Meadow9 iOS & Mobile App Development

Final Report

Toh Kar Le
U2221258J

Supervisor: Dr Chng Eng Siong

College of Computing and Data Science
Nanyang Technological University

Academic Year 2025/2026

Abstract

Long-duration meeting transcription systems have become increasingly prevalent in both enterprise and professional settings. While speech-to-text accuracy has improved substantially, effective and efficient retrieval of information from lengthy transcripts remains a significant challenge. Meadow9+, a long-meeting transcription application developed by ABAX AI, recently introduced a chatbot feature to address this need; however, its current implementation processes entire transcripts directly as input to a large language model (LLM), resulting in high latency, excessive computational cost, and degraded answer quality due to long-context limitations.

This project proposes and develops a Hybrid Retrieval System to improve transcript retrieval and chatbot performance for Meadow9+. The system combines three coordinated subsystems: a summarization pipeline for high-level transcript understanding, a Retrieval-Augmented Generation (RAG) pipeline for grounded conversational question answering, and a keyword-based search workflow for direct date-based, speaker-based, and lexical retrieval. By building a shared retrieval foundation over transcript chunks and summary-derived representations, the proposed system supports both high-level understanding and fine-grained evidence access while reducing inference overhead and extending Meadow9+ beyond chatbot-only interaction. In addition, this work introduces a comprehensive evaluation pipeline that systematically assesses retrieval quality and end-to-end question-answering performance using metrics such as Hit Rate, Mean Reciprocal Rank (MRR), F1 score, and overall accuracy, alongside detailed latency benchmarking.

Experimental results demonstrate that the RAG-based QA achieves comparable accuracy to the existing full-transcript LLM approach while delivering substantial improvements in response time, validating the effectiveness of retrieval-driven optimization. Beyond immediate chatbot acceleration, the implemented architecture also supports transcript summarization and keyword-based search tailored to noisy speech-to-text outputs. This report presents the design, implementation, and evaluation of the overall Hybrid Retrieval System.

Contents

Abstract **1**

1 Introduction **8**
1.1 Background . . . 8
1.2 Motivations and Objectives . . . 8
1.3 Scope . . . 9
1.4 Contributions . . . 10
1.5 Project Schedule . . . 11

2 Literature Review **13**
2.1 Otter.ai . . . 13
2.1.1 Retrieval Strategies . . . 13
2.1.2 Accuracy and Latency Characteristics . . . 15
2.1.3 Transparency of Retrieval and Reasoning . . . 15
2.1.4 Discussion and Implications . . . 15
2.2 Claude Code . . . 16
2.2.1 Retrieval Strategies . . . 16
2.2.2 Accuracy and Latency Characteristics . . . 17
2.2.3 Transparency of Retrieval and Reasoning . . . 17
2.2.4 Discussion and Implications . . . 18
2.3 Pre-Project Meadow9+ . . . 19
2.3.1 Retrieval Strategies . . . 19
2.3.2 Accuracy and Latency Characteristics . . . 20
2.3.3 Transparency of Retrieval and Reasoning . . . 20
2.3.4 Discussion and Implications . . . 21

3 Proposed System Architecture **22**
3.1 System Overview . . . 22
3.2 Indexing Phase . . . 22
3.3 Querying Phase . . . 23
3.4 Hybrid Retrieval System . . . 24
3.5 Evaluation Architecture . . . 24

4 System Implementation **27**
4.1 Implementation Overview . . . 27
4.1.1 Shared Processing Backbone . . . 27
4.1.2 End-to-End Processing Flow . . . 27
4.2 Shared Infrastructure . . . 28

4.2.1 Technology Stack 28
4.2.2 Global Storage Design 28
4.2.3 Shared Metadata Model 28
4.2.4 Role-Specific Model Stack 29
4.3 Summarization Pipeline Implementation 29
4.3.1 Transcript Chunking for Summarization 29
4.3.2 Per-Chunk Structured Extraction 30
4.3.3 Hierarchical Merge and Canonical Summary Schema 30
4.3.4 Summary Embedding Documents 30
4.4 RAG Pipeline Implementation 31
4.4.1 Transcript Chunking and Shared Indexing 31
4.4.2 Hybrid Retrieval and Ranking 32
4.4.3 Context Assembly and Prompt Construction 33
4.5 Keyword-Based Search Implementation 33
4.5.1 Keyword-First Retrieval Design 33
4.5.2 Two-Phase Search Flow 34
4.5.3 Date and Speaker Filtering 34
4.5.4 Relationship to the RAG Subsystem 34
4.6 Deployment Implementation 35
4.6.1 Retrieval Server Deployment Architecture 35
4.6.2 Persistent Runtime Storage 35
4.6.3 End-to-End Deployed Processing Flow 36
4.6.4 Streaming Chat Responses 36
4.6.5 Retrieved Evidence in Chat Responses 36
4.6.6 Operational Interfaces and Health Monitoring 37
4.7 Prototype Flutter Client 37

5 Evaluation and Results **39**
5.1 Evaluation Pipeline Implementation 39
5.1.1 Dataset Generation 39
5.1.2 Evaluation Metrics 40
5.1.3 Speed Benchmarking Design 40
5.2 Experimental Results 41
5.2.1 RAG Quality Metrics 41
5.2.2 Speed Performance 42

6 Conclusion and Future Work **44**
6.1 Conclusion 44
6.2 Future Work 45

6.2.1 Adaptive Routing for Transcript Length 45
6.2.2 Multiple-Language Support 45
6.2.3 Production Hardening and Integration 45
6.2.4 Refinement of Answer Evaluation 46

A Detailed RAG Implementation **47**
A.1 Implemented Retrieval Server Interfaces 47
A.2 Transcript Parsing, Chunking, and Storage 49
A.3 Hybrid Retrieval Internals and Reranking 51
A.4 Context Formatting and Prompt Construction 59

B Detailed Evaluation Pipeline Implementation **64**
B.1 Dataset Generation Artefacts 64
B.2 Metric Implementation Details 66
B.3 Speed Benchmarking Artefacts 67

C Extended Evaluation Examples **69**
C.1 Stage-Separated Diagnostic Scenarios 69
C.1.1 Retrieval Failure 69
C.1.2 Generation Failure 69
C.1.3 Perfect Performance 70
C.2 Detailed Worked Examples 70
C.2.1 Success Case 70
C.2.2 Failure Case 71
C.2.3 False Negative Case 71

Bibliography **76**

List of Figures

1.1 Gantt chart representing the project timeline 11

2.1 Otter.ai iOS application interface showing keyword search, AI chat, and summarization features 14

3.1 System integration: transcript forwarding, summary return, and retrieval indexing 23
3.2 Indexing phase: transcript processing, chunking, and embedding stages . 23
3.3 Querying phase: retrieval and response generation 24
3.4 Querying phase: high-level query-to-response architecture 24
3.5 Evaluation pipeline for the RAG question-answering subsystem 25
3.6 Benchmarking: RAG-based QA subsystem vs. pure LLM baseline 26

4.1 Example canonical summary JSON used as the source structure for summary embedding document generation 31
4.2 Hybrid retrieval, fusion, and reranking in the RAG subsystem 32
4.3 Keyword-based search workflow in the final system 34
4.4 Prototype Flutter client screenshots showing the structured summary tab, Ask AI tab with expandable retrieved sources, and global keyword search interface 38

A.1 SRT input format with sequence numbers and timestamps 49
A.2 Parsed SRT structure with speaker metadata extraction 49
A.3 Token-based merging algorithm for chunk size optimization 50
A.4 Document processing pipeline 50
A.5 Vector store creation from documents 51
A.6 Incremental document addition 51
A.7 ChromaDB storage directory structure 51
A.8 Vector search core function showing semantic retrieval with optional metadata filtering 52
A.9 BM25 tokenization function using lowercase split 52
A.10 BM25 index initialization with corpus tokenization 52
A.11 BM25 query tokenization and metadata filtering 53
A.12 Phonetic word tokenization and filtering 53
A.13 Double Metaphone phonetic encoding 54
A.14 Phonetic encoding examples for homophone matching 54
A.15 Phonetic retrieval SQL query with transcript filtering 54
A.16 Phonetic score conversion and document retrieval 55
A.17 Rare token detection using Zipf frequency 55
A.18 Extracting rare tokens from query 56

A.19 Fuzzy scoring algorithm: setup and initialization 56
A.20 Fuzzy scoring algorithm: nested loop with RapidFuzz ratio 57
A.21 Fuzzy top-k selection and ranking 57
A.22 Reciprocal Rank Fusion weights for the hybrid retrieval signals 58
A.23 Expected reranker response format 58
A.24 Mapping reranked results back to the original documents 59
A.25 Top-n selection after reranking 59
A.26 Chunk formatting with individual metadata headers 60
A.27 Output format example 60
A.28 Citation formatting for user transparency and source attribution 61
A.29 Grounded QA system message template defining assistant role and constraints 62
A.30 Complete RAG chat prompt construction with history support 62
A.31 Chat history structure for multi-turn conversation support 62
A.32 Streaming response handling for real-time token display 63

B.1 Example transformation from raw SRT transcript chunks to a grouped window with temporal metadata 64
B.2 LangChain pipe syntax for QA generation chain 65
B.3 QA generation prompt template with requirements and format 65
B.4 Ground truth JSON format example with timestamps 65
B.5 Interval intersection algorithm for retrieval relevance checking 66
B.6 Text preprocessing for token-level comparison 67
B.7 Precision, recall, and harmonic mean calculation with Counter intersection 67
B.8 LLM judge prompt template for semantic correctness evaluation 67
B.9 Interleaved execution pattern for unbiased speed benchmarking 68

C.1 Retrieval failure scenario diagnostic example 69
C.2 Generation failure scenario with successful retrieval 69
C.3 Perfect performance scenario for both retrieval and generation 70
C.4 Detailed success case example with perfect retrieval and generation . . . 70
C.5 Failure case example with retrieval success but generation error 71
C.6 False negative example with more specific generated answer 71

List of Tables

1.1 Detailed project schedule 12

4.1 Third-party libraries and frameworks categorized by functionality including LangChain, ChromaDB, and retrieval tools 28

5.1 Per-transcript RAG quality metrics aggregated results 41

5.2 Per-transcript speed comparison showing speedup factors and end-to-end accuracy 42

A.1 High-level summary of implemented retrieval server interfaces 48

A.2 Search method characteristics in the hybrid retrieval stage used by the RAG subsystem 58

B.1 Ground truth schema field descriptions 66

B.2 Statistical metrics table for latency analysis 68

B.3 Comparative metrics between RAG and Pure LLM approaches 68

Chapter 1

Introduction

1.1 Background

Large Language Models (LLMs) represent a class of deep learning architectures trained on massive datasets, enabling them to comprehend and generate natural language with high versatility across domains [1]. As of 2025, approximately 67% of organizations worldwide have adopted LLMs to support their operational workflows [2]. One of the most prominent applications of LLMs is in conversational agents or chatbots [3]. These systems often integrate retrieval pipelines [4] that extract domain-specific information from a database or knowledge store, and provide the retrieved content as context to the LLM for response generation. This architecture allows users to query against proprietary or organizational datasets using natural language and receive accurate, context-specific responses.

Meadow9+, a speech recognition application designed to record and transcribe long-duration meetings, is one such platform that integrates an LLM-powered chatbot to enable retrieval of information from meeting transcripts. The platform is widely utilised by corporations, welfare organizations, and medical professionals who rely on accurate transcription of meetings. Since meetings are where critical discussions and decisions take place, these transcripts effectively serve as a knowledge base for the organization. The integrated chatbot therefore plays a crucial role in extracting actionable insights from this knowledge, and the system must ensure both accuracy and responsiveness of the retrieval process.

Currently, the Meadow9+ chatbot processes full transcripts directly as input to the LLM. This design leads to two major limitations. First, excessively long inputs exacerbate the "lost in the middle" phenomenon [5], where the LLM exhibits degraded performance when reasoning over information embedded in the middle sections of extended context windows. Second, longer context sequences increase computational overhead, thereby slowing inference and degrading user experience [6]. Furthermore, transcription outputs often contain inaccuracies and inconsistencies [7], and the current chatbot system lacks functionalities to handle noisy and ambiguous transcript data. These factors reduce usability and limit the effectiveness of the retrieval process for users who need quick and accurate access to meeting information.

1.2 Motivations and Objectives

Meeting transcripts only become truly valuable and actionable when users can quickly locate decisions, tasks, and explanations. However, Meadow9+ currently lacks an effective

retrieval mechanism to support such use cases, which significantly limits the practical utility of long-duration transcripts. As mentioned, the existing Meadow9+ chatbot employs a monolithic prompting approach, in which entire transcripts are provided directly to the LLM. This design leads to high latency, increased computational overhead, and degraded user experience, rendering the system unreliable in real-world scenarios. Improving information retrieval within Meadow9+ therefore directly addresses a critical gap in the current system and has immediate applicability in real-world deployments.

This project presents an opportunity to apply modern information retrieval (IR) and LLM integration techniques beyond naive prompting. In particular, Retrieval-Augmented Generation (RAG) combines classical retrieval methods with generative models, enabling systems to retrieve relevant document segments before generation [8]. At the same time, transcript systems benefit from broader retrieval capabilities that extend beyond question answering, including transcript summarization and direct keyword-based search. The project therefore also enables exploration of advanced retrieval enhancements, including query preprocessing, phonetic encoding, fuzzy matching, summarization, and metadata-aware search.

Motivated by these considerations, this project aims to design and implement a Hybrid Retrieval System that improves transcript understanding, retrieval accuracy, and end-to-end system performance for long meeting transcripts. Specifically, the objectives are:

- Build a transcript retrieval pipeline that indexes long meeting transcripts into retrievable units and supports semantic retrieval via embeddings.
- Integrate retrieval into the chatbot using a RAG workflow to avoid full-transcript prompting, improving latency while maintaining answer quality.
- Implement a summarization pipeline that provides high-level transcript understanding and supports broad-query retrieval.
- Implement metadata-aware keyword-based search to support date-based, speaker-based, and lexical retrieval over the transcript corpus.
- Develop a comprehensive evaluation pipeline to measure retrieval quality and end-to-end question answering performance using standard metrics, including hit-based metrics, reciprocal-rank metrics, and QA-style correctness metrics.
- Conduct empirical benchmarking comparing RAG against the existing pure LLM baseline under realistic conditions, with a focus on both accuracy and response latency.

1.3 Scope

The scope of this project focuses on the design and implementation of a Hybrid Retrieval System tailored for long-duration meeting transcripts. This includes developing appropriate transcript preprocessing strategies, such as chunking and embedding, to enable

efficient and scalable retrieval from large textual documents, as well as building coordinated summarization, conversational QA, and keyword-search capabilities on top of that shared foundation. Particular emphasis is placed on handling the characteristics of long-form transcripts, where individual segments may vary in length and informational density, and ensuring that the resulting representations are suitable for downstream retrieval and generation tasks.

In addition to system design, the project places strong emphasis on quantitative evaluation and performance analysis. Retrieval effectiveness is systematically assessed using established information retrieval metrics, including Hit Rate, Mean Reciprocal Rank (MRR) [9], F1-score, and Accuracy, to provide an objective measure of retrieval and answer quality. Furthermore, speed benchmarking is conducted to evaluate system efficiency, covering both end-to-end response latency and component-level performance across the retrieval and generation pipeline. The scope also includes the implementation of summarization and keyword-search capabilities as part of the final hybrid system, even though those two subsystems are not the primary focus of the comparative benchmark.

Several aspects are explicitly considered out of scope to maintain a clear focus on transcript retrieval and chatbot optimization. Improvements to the underlying Automatic Speech Recognition (ASR) model are excluded, as the project assumes transcripts are provided by the existing Meadow9+ transcription system. Similarly, user interface and user experience (UI/UX) redesign of the Meadow9+ application is not addressed. Finally, the project does not involve training or fine-tuning custom large language models, and instead focuses on optimizing system-level integration and retrieval strategies using pre-existing LLMs.

1.4 Contributions

Note that this section only provides an overview of the key contributions made. Additional details will be discussed in subsequent chapters, where they are grouped and presented differently. A summary of key contributions to this project is presented below:

Developed a shared hybrid retrieval foundation that indexes long-form meeting transcripts using token-based chunking and combines semantic, lexical search, phonetic search, and fuzzy matching. The pipeline employs Reciprocal Rank Fusion (RRF) [10] and cross-encoder reranking [11] to surface highly relevant chunks, providing the common retrieval backbone used by the summarization, RAG, and keyword-search subsystems.

Integrated the retrieval foundation into a RAG-based QA subsystem using LangChain orchestration, replacing full-transcript prompting with context-aware retrieval. This approach leverages per-transcript metadata filtering and temporal alignment, delivering up to 8.35× speedup over the pure LLM baseline, directly addressing latency and token cost constraints in production conversational interfaces.

Implemented a hierarchical summarization subsystem that generates structured transcript summaries and summary-derived retrieval documents. This adds high-level transcript understanding to the system and enables summary information to work hand-

in-hand with the QA subsystem for vague or broad questions.

Implemented a global keyword-first search subsystem tailored to noisy ASR transcripts, enabling efficient date-based, speaker-based, and keyword-based retrieval in addition to chatbot-driven querying. The system employs a two-phase pipeline that ranks transcripts globally before running per-transcript chunk retrieval with cross-encoder reranking, with phonetic and fuzzy matching to handle ASR transcription noise.

Implemented a comprehensive evaluation framework measuring both retrieval quality (Hit Rate, MRR) and generation quality (F1 score, LLM-as-judge accuracy). The framework validates temporal overlap between retrieved chunks and ground-truth spans, providing multi-dimensional assessment of the RAG subsystem. Evaluation on 390 automatically generated QA pairs demonstrates 77.5% semantic correctness.

Conducted controlled empirical benchmarking comparing the RAG subsystem against a pure LLM baseline under realistic conditions. Component-level timing measurements isolate retrieval overhead from generation time, and interleaved query execution mitigates prompt caching bias. Results demonstrate that the RAG subsystem achieves 77.5% accuracy at up to 8.35× speedup compared to baseline, establishing practical viability for real-time transcript question-answering in resource-constrained environments.

1.5 Project Schedule

The detailed project schedule is presented in Table 1.1, with the corresponding timeline visualization shown in Figure 1.1.

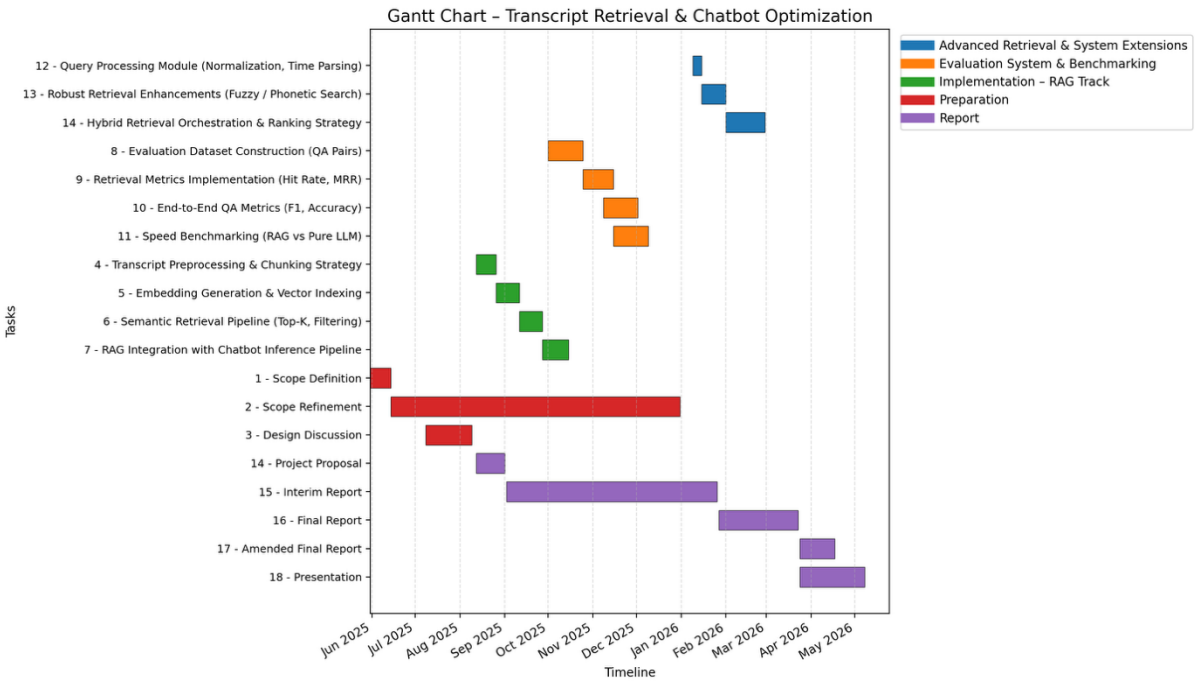

Figure 1.1: Gantt chart representing the project timeline

Index	Task Name	Start Date	End Date
Preparation			
1	Scope Definition	31 May 2025	14 Jun 2025
2	Scope Refinement	14 Jun 2025	31 Dec 2025
3	Design Discussion	8 Jul 2025	9 Aug 2025
Implementation – RAG Track			
4	Transcript Preprocessing & Chunking Strategy	12 Aug 2025	26 Aug 2025
5	Embedding Generation & Vector Indexing	26 Aug 2025	11 Sept 2025
6	Semantic Retrieval Pipeline (Top-K, Filtering)	11 Sept 2025	27 Sept 2025
7	RAG Integration with Chatbot Inference Pipeline	27 Sept 2025	15 Oct 2025
Evaluation System & Benchmarking			
8	Evaluation Dataset Construction (QA Pairs)	1 Oct 2025	25 Oct 2025
9	Retrieval Metrics Implementation (Hit Rate, MRR)	25 Oct 2025	15 Nov 2025
10	End-to-End QA Metrics (F1, Accuracy)	8 Nov 2025	2 Dec 2025
11	Speed Benchmarking (RAG vs Pure LLM)	15 Nov 2025	9 Dec 2025
Advanced Retrieval & System Extensions			
12	Query Processing Module (Normalization, Time Parsing)	9 Jan 2026	15 Jan 2026
13	Robust Retrieval Enhancements (Fuzzy / Phonetic Search)	15 Jan 2026	1 Feb 2026
14	Hybrid Retrieval Orchestration & Ranking Strategy	1 Feb 2026	28 Feb 2026
Report			
15	Project Proposal	12 Aug 2025	1 Sept 2025
16	Interim Report	2 Sept 2025	26 Jan 2026
17	Final Report	27 Jan 2026	23 Mar 2026
18	Amended Final Report	24 Mar 2026	17 Apr 2026
19	Presentation	24 Mar 2026	8 May 2026

Table 1.1: Detailed project schedule

Chapter 2

Literature Review

In today's competitive landscape, modern transcription-based applications and meeting assistants increasingly integrate chat-based interfaces to allow users to query recorded conversations. In approaching the development of this project, a comprehensive review was conducted on the retrieval capabilities of three existing products: Otter.ai, Claude Code and the existing Meadow9+ mobile application. These products were selected due to their relevance to the proposed project.

The literature review focused on three key aspects: retrieval strategies, accuracy and latency characteristics, and transparency of retrieval and reasoning. Instead of trying to replicate existing solutions, this review aimed to understand how these platforms effectively address retrieval challenges and identify opportunities for improvement in developing our project. An in-depth analysis of the technological stacks and architectures will be presented in later chapters, specifically in the context of Meadow9+, as such information is not publicly available for the other products reviewed.

This chapter is divided into three main sections. Section 2.1 reviews Otter.ai, and Section 2.2 examines Claude Code, focusing exclusively on their retrieval system. Section 2.3 evaluates the Meadow9+ mobile application prior to this project.

2.1 Otter.ai

Otter.ai is a widely adopted speech-to-text and meeting assistant platform that provides real-time transcription, post-meeting summaries, and multiple mechanisms for retrieving information from recorded conversations [12]. As a mature commercial system, it represents a strong baseline for evaluating existing approaches to transcript retrieval and conversational interaction.

Otter.ai caters to a wide range of users, including businesses and students [13, 14]. While it is primarily used for transcribing meetings, the platform also proves valuable for sales calls and lecture notes [13, 14]. While Otter.ai offers applications on different platforms including web, iOS and Android [15], this discussion will focus exclusively on the iOS application.

2.1.1 Retrieval Strategies

Otter.ai provides three primary mechanisms for retrieving information from transcripts: keyword-based search, LLM-powered conversational querying (Otter AI Chat), and LLM-Based Summarization, as illustrated in Figure 2.1 [12, 15].

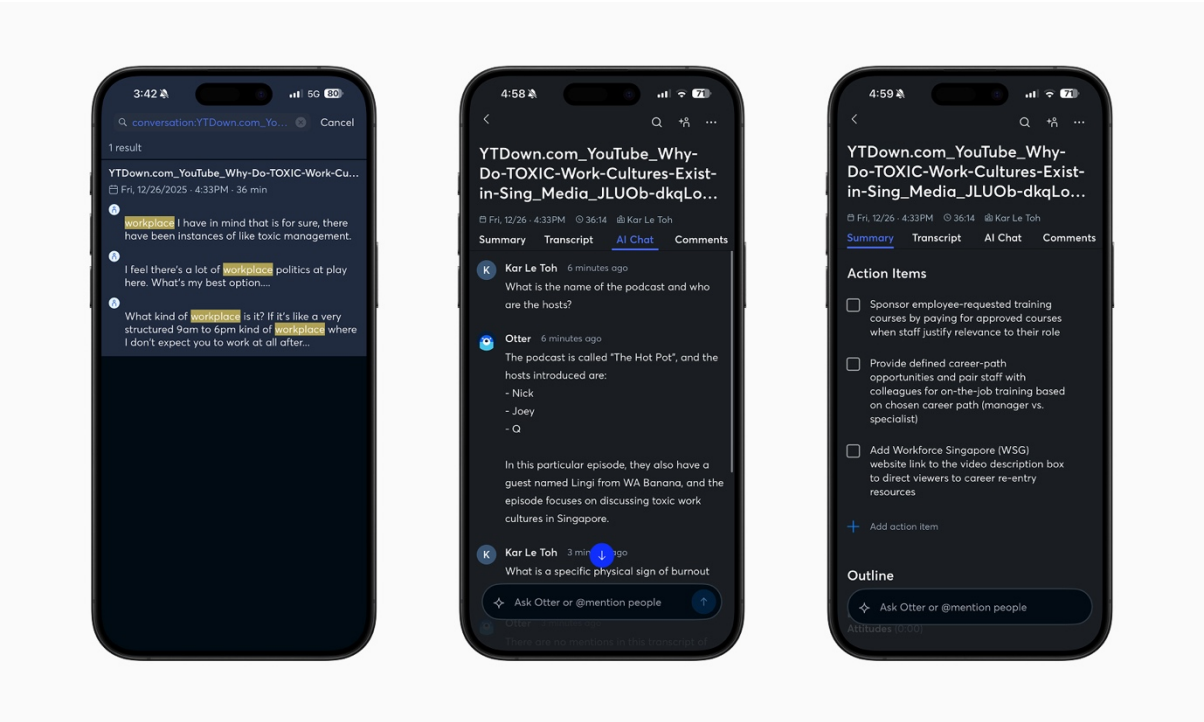

Figure 2.1: Otter.ai iOS application interface showing keyword search, AI chat, and summarization features

Keyword-Based Retrieval

Otter.ai supports keyword-based search over notes and transcripts [15]. Although Otter.ai does not publicly disclose the exact retrieval algorithm used, empirical testing suggests that the system employs a term-weighted ranking function such as TF-IDF or BM25. For example, queries containing additional discriminative terms such as "toxic workplace" versus "workplace" significantly alter result rankings in a manner consistent with BM25-style term frequency saturation and inverse document frequency weighting [16]. In addition, Otter.ai supports date-based filters over transcript metadata [15]. This feature substantially improves usability and demonstrates the value of combining structured metadata filtering with unstructured text search.

LLM-Based Conversational Retrieval (Otter AI Chat)

Otter AI Chat allows users to ask free-form natural language questions either across all transcripts or within a specific transcript. While Otter.ai does not explicitly disclose its backend architecture, its FAQ indicates that transcript content and prompts are transmitted to third-party services only when users actively interact with Otter AI Chat, strongly suggesting reliance on externally hosted LLMs from third-party providers [17].

Behavioral observations indicate that the global chat interface likely employs a retrieval step to narrow down relevant transcripts before passing context to the LLM, as responses often include links to specific conversations. This behavior is consistent with a RAG

pipeline operating at the document level. In contrast, the per-transcript chat interface appears to process the entire transcript holistically, as responses rarely reference precise supporting segments or timestamps.

Notably, Otter AI Chat responses tend to be abstractive and conversational, favoring rephrased summaries over verbatim quotations. This design choice may be driven by prompt engineering rather than retrieval limitations; however, it also suggests limited emphasis on fine-grained, chunk-level semantic retrieval at query time.

LLM-Based Summarization

Beyond interactive retrieval, Otter.ai generates an extensive per-transcript summary using LLM-based processing [12]. This summary is structured into three layers: (i) a high-level overview of the meeting, (ii) extracted action items, and (iii) a hierarchical outline organized by timestamps, with further speaker-segmented summaries nested within each time interval. This multi-level summarization demonstrates effective use of LLMs to transform long-form transcripts into structured, navigable representations.

2.1.2 Accuracy and Latency Characteristics

From manual evaluation, Otter.ai's keyword-based search exhibits both high accuracy and negligible latency, consistent with expectations for traditional lexical retrieval systems. This makes keyword search the most reliable and responsive mechanism for precise information lookup. Otter AI Chat, when applied to transcripts up to approximately two hours in length, also demonstrates low perceived latency and strong answer accuracy. This performance is likely enabled by the use of large, cloud-hosted LLMs with sufficient context windows and optimized inference infrastructure.

2.1.3 Transparency of Retrieval and Reasoning

A notable limitation of Otter AI Chat is its lack of transparency. The system provides minimal visibility into which transcript segments support a given answer, and users cannot inspect or verify the retrieved evidence. This opacity limits user trust and makes error diagnosis difficult, particularly in professional or compliance-sensitive settings.

2.1.4 Discussion and Implications

Overall, Otter.ai exemplifies a feature-rich and well-integrated retrieval ecosystem for meeting transcripts. It combines global and local keyword search, conversational querying, and comprehensive summarization to support diverse user workflows. These capabilities collectively demonstrate that effective transcript systems must treat retrieval as a first-class design concern, rather than a secondary feature layered on top of transcription.

However, the apparent lack of fine-grained, chunk-level grounding and the absence of explicit evidence attribution indicate that Otter.ai's system prioritizes fluency and abstraction over verifiability. While this design choice may be acceptable for general meeting summarization, it introduces risks of hallucination and reduces user trust in scenarios where factual accuracy and traceability are critical. This limitation is particularly relevant to Meadow9+, since it cannot rely on large cloud-hosted frontier models with extensive context windows and optimized inference pipelines. Instead, it employs a smaller, self-hosted LLM with stricter latency and context constraints. Consequently, passing full transcripts, or even large document-level contexts, into the model is neither scalable nor reliable.

These observations collectively imply that Meadow9+ must adopt a *more retrieval-centric and evidence-grounded design* than Otter.ai. Rather than treating retrieval as a preliminary filtering step, retrieval must be a first-class system component that operates at multiple granularities (document-level, chunk-level, and temporal level).

In contrast, Meadow9+ is designed to decouple retrieval from generation and to emphasize transparent, inspectable retrieval results. This includes (i) hybrid lexical–semantic retrieval to handle both exact terms and paraphrased queries, (ii) phonetic and fuzzy matching to mitigate transcription errors, and (iii) chunk-level evidence selection to reduce prompt length. By explicitly surfacing retrieved segments and their timestamps, Meadow9+ aims to improve both answer accuracy and user trust, which is an aspect largely absent in Otter.ai's conversational interface.

2.2 Claude Code

Recent advances in LLM–based agents have extended beyond conversational assistants into domain-specialized systems that tightly integrate retrieval, reasoning, and action. One notable example is Claude Code, a command-line interface (CLI) agent developed by Anthropic that brings LLM-driven reasoning directly into the software development workflow [18]. Claude Code is widely regarded as one of the most capable coding agents currently available, not solely due to model quality, but because of its system-level retrieval and context management design that enables effective reasoning over large, evolving codebases.

2.2.1 Retrieval Strategies

Claude Code adopts an agentic retrieval paradigm, where the LLM actively searches and retrieves relevant context from the environment at query time rather than relying on a precomputed embedding index or static vector store. Specifically, Claude Code is granted controlled read access to the local project directory and retrieves information by invoking standard developer tools such as file search and content matching when needed [19, 20]. Retrieved files or code snippets are then injected directly into the model's context window.

Importantly, retrieval is performed dynamically through tool calls that operate over the

source-of-truth codebase rather than through a precomputed embedding index [19, 20]. This design avoids issues associated with stale embeddings, incomplete indexing, and semantic drift, ensuring that the retrieved context always reflects the most recent state of the project.

Claude Code also supports persistent project-level memory via a lightweight mechanism: a configuration file (e.g., CLAUDE.md) that stores high-level instructions, architectural conventions, and recurring commands. This file is automatically injected into every session, acting as a form of long-term context without requiring a separate memory model or database [21].

2.2.2 Accuracy and Latency Characteristics

The agentic retrieval strategy in Claude Code introduces clear trade-offs between accuracy and latency. On the one hand, dynamically searching the codebase yields high retrieval precision, as the agent can iteratively refine its search queries and inspect exact source files relevant to the task [19]. This approach can improve task success rates in complex, multi-file coding scenarios compared to static retrieval pipelines.

On the other hand, retrieval incurs non-trivial overhead. Each file read and search operation increases prompt size and introduces runtime latency due to tool execution and additional token processing. This design typically leads to higher token usage and slower individual responses compared to systems that rely on pre-indexed embeddings, but the overhead can be acceptable given the resulting improvement in task accuracy and robustness.

To further improve accuracy on complex tasks, Claude supports an extended thinking mode in which the model spends more time deliberating before producing an answer [22]. While this increases latency, it can improve solution quality for tasks requiring multi-step reasoning or global code understanding.

2.2.3 Transparency of Retrieval and Reasoning

A defining characteristic of Claude Code is its emphasis on transparent retrieval and execution. Tool use is permission-gated, and any action that may modify the system requires user approval [19, 20]. This design allows developers to observe how context is retrieved and to intervene when necessary.

Claude Code also provides a "Plan Mode," in which the agent analyzes the codebase and proposes a sequence of actions without executing them. This separation of planning and execution improves trust and enables users to validate the agent's understanding before committing changes [18].

From a system-design perspective, Claude Code demonstrates that retrieval transparency is not only a safety feature but also a debugging aid, allowing users to reason about failures in retrieval or context selection.

2.2.4 Discussion and Implications

Although Claude Code operates in the domain of software engineering, its retrieval design offers several transferable insights for transcript-based systems such as Meadow9+.

First, Claude Code's avoidance of static embedding-based retrieval highlights a broader design principle: retrieval systems must align with the nature of the underlying data. Claude Code adopts a fully agentic retrieval approach precisely because of the characteristics of its target domain. Dynamic, query-time search over the source-of-truth codebase ensures that retrieved context remains up-to-date and semantically precise, avoiding issues associated with stale or incomplete indices.

In contrast, Meadow9+ operates over transcript data that is static once transcription is completed. While a full agentic retrieval paradigm is powerful, it is not strictly required in this setting. Additionally, practical system constraints must also be considered, as Meadow9+ employs a smaller self-hosted LLM with substantially weaker reasoning and tool-use capabilities compared to commercial frontier models used in systems such as Claude Code. Under these constraints, fully agentic retrieval may introduce performance bottlenecks, increased latency, or even unstable retrieval behavior, particularly when iterative reasoning and tool invocation are required.

These observations suggest that the optimal retrieval strategy for Meadow9+ lies in a hybrid approach, combining embedding-based semantic retrieval with symbolic techniques. Such a design leverages the static nature of transcript data to enable efficient indexing while remaining robust to transcription noise and lexical variation. At the same time, it avoids the computational and reasoning overheads associated with fully agentic retrieval, making it more suitable for deployment with resource-constrained, self-hosted LLMs.

Second, Claude Code's trade-off between accuracy and latency offers guidance for Meadow9+'s system design. Claude Code deliberately trades off latency and token efficiency in exchange for higher task accuracy. Each retrieval step incurs additional overhead due to tool invocation, file inspection, and prompt expansion. This trade-off is acceptable in developer workflows, where correctness is often prioritized over response time and where commercial frontier models with strong reasoning capabilities are available.

Meadow9+, however, operates under stricter latency and cost constraints. The system also relies on a self-hosted, smaller-scale LLM, which is significantly less capable than proprietary frontier models. Under such constraints, a fully agentic retrieval loop may introduce excessive latency and unstable performance. Moreover, iterative retrieval decisions place additional reasoning burden on the model itself, increasing the risk of suboptimal search strategies and error propagation.

These observations suggest that the retrieval strategy for Meadow9+ should be front-loaded into deterministic or weakly learned components, such as BM25 and dense embeddings, rather than delegated to the LLM. This reduces the reasoning load on the model and yields more predictable latency characteristics.

Finally, Claude Code's emphasis on transparent retrieval suggests an additional opportunity for Meadow9+. While Meadow9+ targets non-technical users, partial transparency,

such as highlighting which transcript segments were used to generate an answer, can improve user trust and perceived reliability. This is especially important in professional meeting contexts where incorrect attribution or hallucinated decisions can have serious consequences.

Incorporating lightweight transparency mechanisms aligns with Claude Code's philosophy that retrieval visibility is not merely a safety feature but a debugging and validation aid. Such features can also support iterative refinement of retrieval strategies during system evaluation.

2.3 Pre-Project Meadow9+

Meadow9+ is a mobile speech-to-text application developed by researchers from the NTU Speech and Language Laboratory, with initial development dating back to approximately 2021 [23, 24]. It is a product of ABAX AI, a company specializing in advanced speech recognition technologies tailored for enterprise and professional use cases [24, 25].

The platform is primarily designed for Singapore-based business professionals, with a strong emphasis on one-to-one meetings as its core usage scenario [23–25]. Nevertheless, Meadow9+ also supports a broader range of transcription contexts, including multi-party discussions and news audio recordings. A key strength of the system lies in its ability to handle Singaporean accents and domain-specific terminology, addressing a notable gap in the local speech recognition market where generic transcription systems often underperform.

While Meadow9+ demonstrates strong transcription quality, particularly in localized linguistic contexts, its post-transcription information retrieval and interaction capabilities remain limited. The following subsections analyze the retrieval and summarization mechanisms implemented in the pre-project version of Meadow9+, highlighting their strengths and limitations.

2.3.1 Retrieval Strategies

Meadow9+ provides two primary mechanisms for retrieving information from transcripts: (1) text-matching search and (2) LLM-based conversational querying.

Text-Matching Search

The system supports a text-matching search functionality at the global level, allowing users to search across conversation titles. At query time, the search matches user-provided keywords against stored conversation titles, returning transcripts whose metadata contains the queried terms. This design emphasizes simplicity and responsiveness, making it suitable for lightweight retrieval scenarios where meaningful titles are consistently available.

LLM-Based Conversational Retrieval

Meadow9+ integrates an LLM-powered chatbot that allows users to pose free-form natural language questions within a selected transcript [26]. For privacy and data governance considerations, the chatbot relies on a self-hosted LLM.

In the pre-project system, the chatbot passes the full transcript directly into the LLM as input context at query time, without an intermediate retrieval or context selection stage. Responses generated by the system are primarily abstractive and conversational in nature, focusing on synthesized summaries rather than direct quotations or fine-grained references to specific transcript segments.

LLM-Based Summarization

In addition to interactive querying, Meadow9+ generates a per-transcript summary using LLM-based processing [26]. This summary provides a condensed overview of the meeting content and is generated independently of user queries. The summarization output is static and intended for post-meeting review rather than targeted information retrieval. It does not support interaction, filtering, or alignment with subsequent user questions.

2.3.2 Accuracy and Latency Characteristics

Based on manual evaluation, Meadow9+'s text-matching search demonstrates high accuracy and minimal latency under ideal conditions. When relevant metadata is present, search results are returned promptly.

In contrast, the chatbot exhibits high perceived latency and weak answer accuracy when applied to transcripts exceeding approximately 1 hour in length. These performance issues are likely attributable to two factors: (1) the use of a relatively small self-hosted LLM with limited reasoning capacity compared to commercial frontier models, and (2) inefficient utilization of the model's context window due to the absence of retrieval or context selection mechanisms.

2.3.3 Transparency of Retrieval and Reasoning

The pre-project Meadow9+ chatbot presents users with a single generated response per query. The system does not provide explicit indicators of which transcript segments were most relevant to the answer, nor does it surface supporting excerpts or references from the transcript. As a result, users interact with the chatbot as a black-box conversational interface, without visibility into intermediate reasoning or evidence selection processes.

2.3.4 Discussion and Implications

The pre-project Meadow9+ system demonstrates a clear separation between transcription quality and post-transcription interaction capabilities. While the platform is optimized for accurate speech recognition in localized contexts, its retrieval and interaction mechanisms are comparatively lightweight and monolithic, exhibiting several structural limitations that impact effectiveness, scalability, and user experience.

For text-matching search, the reliance on transcript-level metadata significantly constrains retrieval coverage. Because conversation titles and descriptions are optional, many transcripts lack sufficiently descriptive metadata, reducing the practical utility of this search mechanism. Moreover, the absence of keyword-based searching within transcript content prevents users from retrieving specific terms, entities, or discussion points embedded in the conversation itself. The system also does not support natural language or temporal constraints (e.g., "last Friday" or "earlier in the meeting"), which are increasingly expected in modern retrieval interfaces.

The LLM-based conversational retrieval approach addresses some expressiveness limitations by allowing free-form queries; however, its design introduces different trade-offs. Passing entire transcripts directly to the LLM leads to high inference latency for long-duration meetings and exacerbates the lost-in-the-middle problem. Additionally, the abstractive nature of responses, combined with the lack of explicit evidence grounding, reduces answer verifiability and makes systematic error analysis difficult.

Collectively, these observations highlight a fundamental architectural gap in the pre-project Meadow9+ system: the absence of an explicit, fine-grained retrieval layer that mediates between raw transcripts and language model generation. This gap motivates the adoption of a structured RAG framework in the proposed system. By decoupling retrieval from generation and incorporating hybrid retrieval strategies tailored to noisy speech-to-text data, the redesigned system aims to improve latency, accuracy, transparency, and scalability while preserving conversational usability.

Chapter 3

Proposed System Architecture

3.1 System Overview

The proposed system extends the Meadow9+ transcript assistant from a chatbot-centric design into a Hybrid Retrieval System. Instead of relying on a single interaction mode that faces scalability and latency limitations, the redesigned system supports three complementary capabilities over the same transcript corpus: a summarization pipeline for high-level transcript understanding, a retrieval-augmented generation (RAG) pipeline for conversational question answering, and a keyword-based search interface for direct snippet retrieval. [4]

To minimise disruption to the existing Meadow9+ infrastructure, the new Hybrid Retrieval System is deployed as a separate retrieval server that interacts with the current Meadow9+ backend server. This separation reduces the extent of modifications required on the production backend, while enabling the possibility of controlled A/B testing in production to compare the performance of the existing chatbot approach against the new proposed system in this project.

At a high level, the system operates in two distinct phases: an indexing phase and a querying phase.

3.2 Indexing Phase

During the indexing phase, when a user submits a new recording and a corresponding transcription is generated, the transcript continues to be stored in the existing Meadow9+ database as per the current implementation. In addition, the same transcript is forwarded to the Retrieval Server, as illustrated in Figure 3.1.

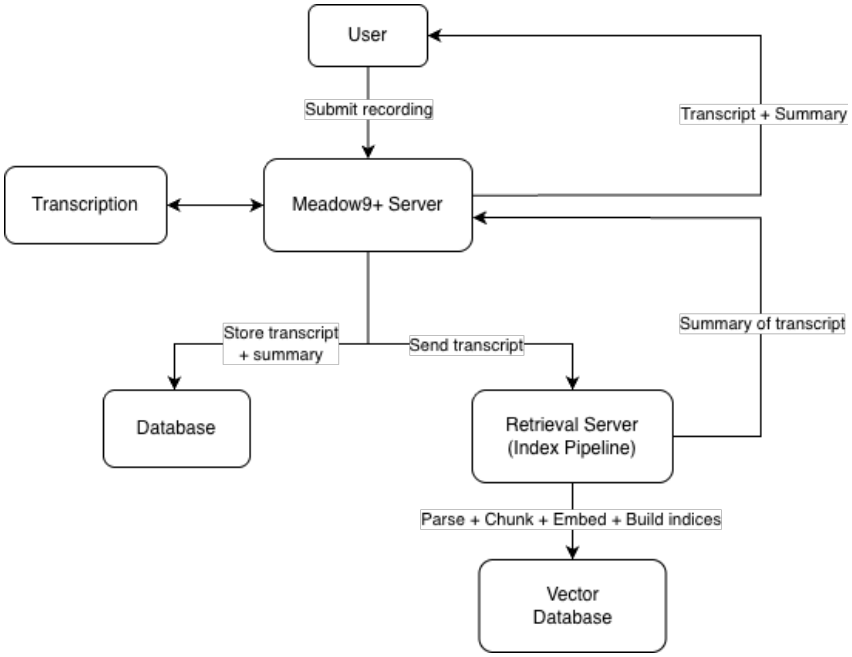

Figure 3.1: System integration: transcript forwarding, summary return, and retrieval indexing

The Retrieval Server performs two tasks in parallel. First, it generates a raw transcript summary and returns it to the Meadow9+ Server. This summary provides high-level abstractions of the transcript, including overview, topic, outline, segment, action, and entity-level information. The Meadow9+ Server can then store this returned summary in the database and return it to the user as a high-level summary of the transcript.

Second, the Retrieval Server performs the processing, chunking, and embedding stages required to prepare the transcript for later retrieval. Figure 3.2 shows this high-level indexing pipeline, where the raw transcript is processed and transformed into indexed representations that can later support retrieval-augmented question answering and search.

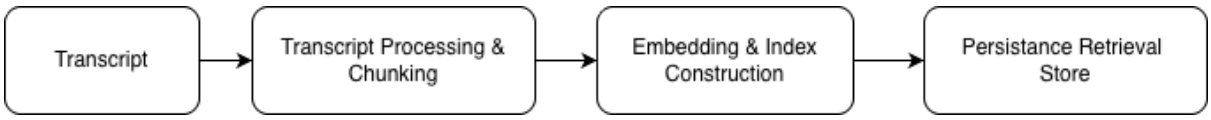

Figure 3.2: Indexing phase: transcript processing, chunking, and embedding stages

3.3 Querying Phase

During querying, user questions are transformed into embeddings, used to retrieve a small set of relevant transcript segments, and supplied to the LLM as contextual grounding for response generation. This query workflow is depicted in Figure 3.3, with the complete RAG pipeline architecture shown in Figure 3.4. [4]

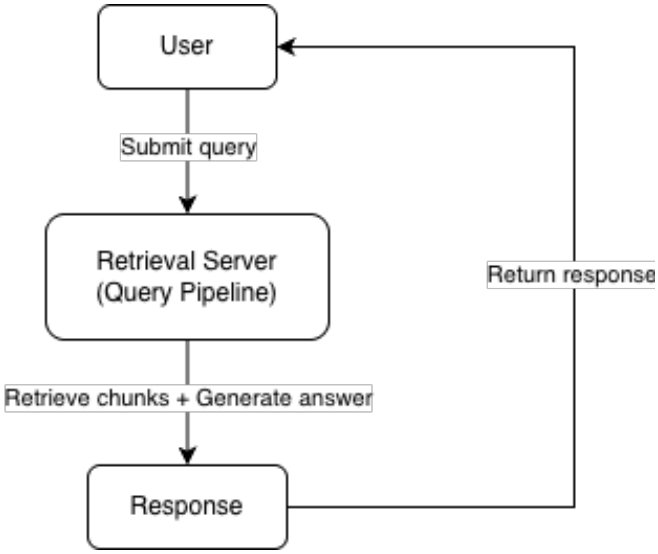

Figure 3.3: Querying phase: retrieval and response generation

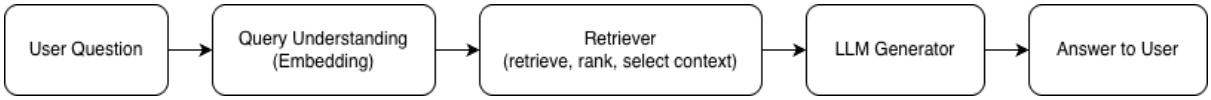

Figure 3.4: Querying phase: high-level query-to-response architecture

3.4 Hybrid Retrieval System

This architecture supports a Hybrid Retrieval System consisting of three complementary components. The first component is the Summarization Pipeline, which provides users with high-level transcript understanding by generating transcript summaries from newly uploaded transcripts. This pipeline is also crucial because the produced summary works hand-in-hand with the RAG pipeline to support more vague, high-level questions that benefit from transcript-level framing rather than only fine-grained evidence retrieval. The second component is the RAG Pipeline, which supports conversational question answering by retrieving relevant transcript evidence and grounding LLM responses in that retrieved context. The third component is Keyword-Based Search, which enables faster direct retrieval for keyword-based, date-based, and speaker-based lookup tasks.

Together, these three components enable the system to support both high-level understanding and fine-grained retrieval over long transcripts. The detailed implementation of these components will be discussed in Chapter 4.

3.5 Evaluation Architecture

The current evaluation architecture focuses on the conversational QA path of the RAG subsystem. At a high level, the system evaluates retrieval and answer generation using

a dedicated question–answer dataset derived from transcript data, then runs batch evaluation to measure how effectively the system retrieves relevant evidence and produces grounded answers.

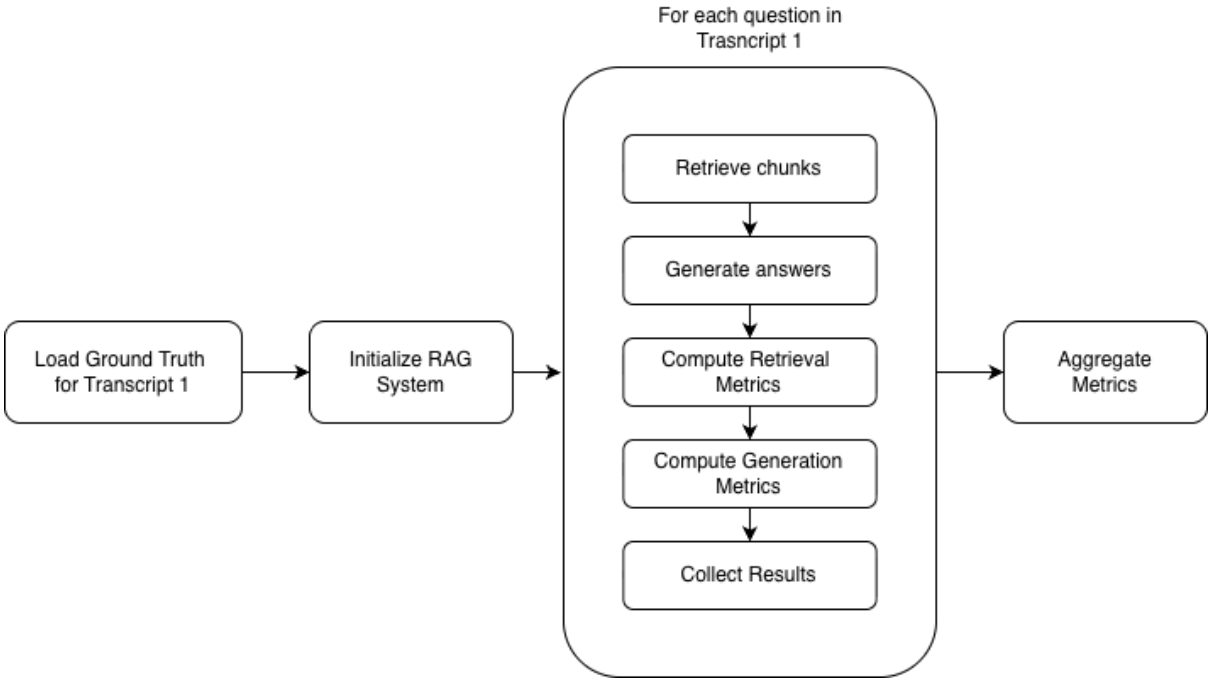

Figure 3.5: Evaluation pipeline for the RAG question-answering subsystem

The evaluation outputs include retrieval metrics, generation metrics, and speed benchmarking. Retrieval quality is assessed using ranking-based measures such as hit rate and mean reciprocal rank (MRR), while generation quality is assessed using lexical and semantic correctness measures. [9, 27] In addition, the system benchmarks the RAG-based QA subsystem against a pure-LLM baseline to compare latency and overall response efficiency across different transcript lengths, as shown in Figure 3.6.

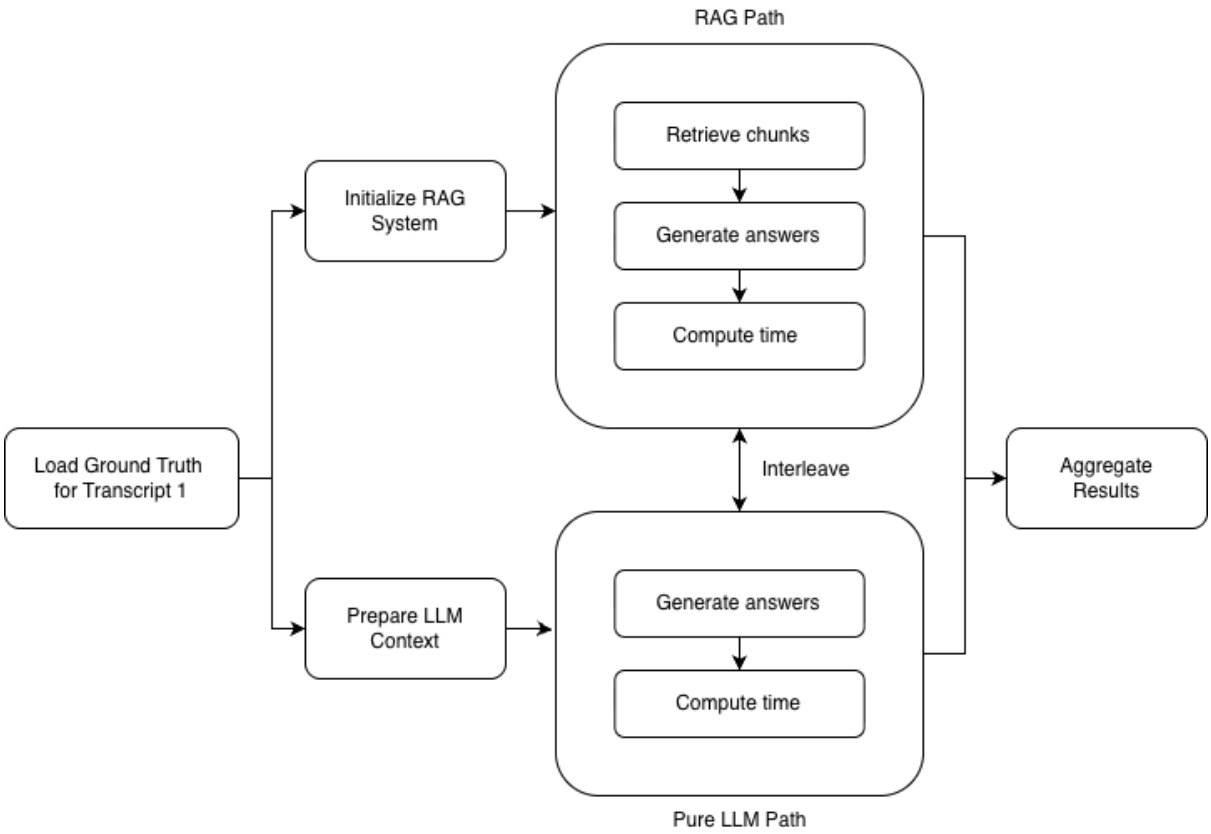

Figure 3.6: Benchmarking: RAG-based QA subsystem vs. pure LLM baseline

This evaluation architecture focuses on the RAG pipeline, as this is the subsystem that can be directly compared against the existing chatbot in terms of performance, scalability, and latency. The summarization pipeline and keyword-based search interface are presented as implemented capabilities of the proposed system, but they are not the primary focus of the comparative evaluation.

Chapter 4

System Implementation

This chapter presents the implementation of the Hybrid Retrieval System. The final system comprises three coordinated subsystems built on shared transcript ingestion and retrieval infrastructure: a summarization pipeline for high-level understanding, a RAG pipeline for grounded question answering, and a keyword-based search workflow for direct snippet retrieval.

4.1 Implementation Overview

4.1.1 Shared Processing Backbone

The final implementation was designed around one shared transcript-processing backbone rather than three completely independent systems. Each uploaded transcript is parsed once, enriched with timestamps and speaker metadata, and stored in a global retrieval layer. On top of this backbone, the system produces both fine-grained transcript chunks and summary-derived documents, allowing the same underlying corpus to support detailed QA, broad topic-level retrieval, and search-oriented lookup.

This architecture reduced duplicated infrastructure and made the three subsystems mutually reinforcing. In particular, the summary pipeline does not only generate a user-facing summary; it also produces retrieval artefacts that improve broad-query handling in the chatbot and transcript ranking in the search bar.

4.1.2 End-to-End Processing Flow

The implementation flow follows a consistent progression from raw transcript to user-facing capability. First, incoming SRT transcripts are parsed into structured segments with timestamps and speaker annotations. Second, those segments are merged into retrieval-sized chunks and transformed into document objects for embedding and storage. Third, the same transcript is passed through a summarization path that produces one canonical summary and multiple summary-derived retrieval documents. Finally, the chatbot and search bar query the shared retrieval layer with different ranking and output strategies. This shared flow is the main reason the final system can support three different interaction styles without maintaining three separate backends.

4.2 Shared Infrastructure

4.2.1 Technology Stack

The implementation uses LangChain for document abstraction and prompt orchestration, FastAPI and Pydantic for service-layer interfaces, Chroma for persistent vector storage, and a locally hosted model stack for embedding, reranking, summarization, and answer generation. [28–36]

Category	**Key Libraries**	**Primary Purpose**
Framework	LangChain, FastAPI, Pydantic	RAG orchestration and REST API
Vector Store	Chroma	Persistent embedding storage and similarity search
Retrieval	rank-bm25, metaphone, rapidfuzz, wordfreq	Hybrid search with phonetic and fuzzy matching
LLM Integration	langchain-openai	OpenAI-compatible API wrapper for local/cloud models
Data Processing	dateparser, pandas, numpy, matplotlib	Natural language parsing and data analysis
Development	pytest, requests	Testing and external service communication

Table 4.1: Third-party libraries and frameworks categorized by functionality including LangChain, ChromaDB, and retrieval tools

4.2.2 Global Storage Design

All transcript chunks are stored in a single global index rather than separate transcript-specific indices. This simplifies deployment, supports cross-transcript retrieval, and allows transcript scoping through metadata filters. The same storage layer also contains summary-derived documents, so broad transcript-level evidence and low-level chunk evidence can be retrieved through one consistent interface. [37, 38]

4.2.3 Shared Metadata Model

The same metadata model is reused across subsystems. Transcript identifiers, timestamps, speaker information, and upload-date fields are attached during ingestion and then reused for transcript-scoped retrieval, user-facing citations, summary-document generation, and keyword-based filtering. This was an important implementation choice because it allowed search, summarization, and QA to operate over one coherent document representation instead of maintaining separate data-preparation logic for each feature.

4.2.4 Role-Specific Model Stack

The model stack is also separated by role rather than handled by one generic LLM endpoint. Embeddings are generated by a dedicated locally-hosted jina-embeddings-v3-f16 model because retrieval quality depends on stable dense representations rather than generative behavior. Reranking is handled by a separate cross-encoder bge-reranker-v2-m3-FP16 model because pairwise relevance scoring has different accuracy and latency requirements from both embeddings and answer generation. The local Qwen3-14B-Q4 model is then reused for the two generative tasks in the system, namely summarization and grounded answer generation. For evaluation pipelines, DeepSeek, via the DeepSeek API, is used to generate ground-truth data.

All local models are hosted using llama.cpp server, which provides efficient inference with optimized CPU/GPU acceleration and OpenAI-compatible API endpoints. [33–36, 39] These models are hosted as separate services to better simulate a production deployment environment, where different inference components may be scaled, tuned, or monitored independently. This separation also makes model replacement easier, because any one component can be swapped out when a better embedding model, reranker, or generator becomes available without requiring the entire pipeline to be redesigned.

4.3 Summarization Pipeline Implementation

The summarization pipeline forms the first subsystem of the Hybrid Retrieval System. It was added to support high-level transcript understanding and to provide transcript-level retrieval artefacts for broad or vague downstream queries. A single full-context LLM call is not reliable for long transcripts, so the implementation adopts a hierarchical multi-stage design in which summarization is decomposed into chunking, per-chunk extraction, and global synthesis.

4.3.1 Transcript Chunking for Summarization

The summarization path first splits the transcript into token-counted chunks rather than relying on a simple character limit. Chunk boundaries are aligned as far as possible with speaker turns so that local summaries preserve conversational flow, participant structure, and topic boundaries. Small overlaps are carried across adjacent chunks to reduce information loss at chunk boundaries, which is particularly important for discussions that span multiple turns or contain deferred references.

This chunking strategy differs in purpose from transcript chunking for retrieval. In the summarization subsystem, chunking exists to keep each extraction request within the local model's practical input budget while preserving enough local coherence for reliable structured extraction.

4.3.2 Per-Chunk Structured Extraction

Each chunk is then processed asynchronously by an LLM to generate a structured JSON-style chunk summary. Each chunk summary captures local discussion points, entities, and action items, forming an intermediate representation that is easier to merge than raw transcript text.

4.3.3 Hierarchical Merge and Canonical Summary Schema

After chunk extraction, the intermediate summaries are merged into a canonical transcript-level summary.

The resulting canonical summary is stored in structured JSON form. Its fields capture transcript-level framing information such as participants, conversation type, purpose, overview, key topics, detailed topics, recurring themes, outline sections, notable segments, action items, and entities. This schema is richer than a simple free-text abstract and is designed to support both user-facing summarization and downstream retrieval over specific summary sections.

4.3.4 Summary Embedding Documents

The summarization subsystem does not stop at producing one canonical summary JSON file. During ingestion, the system also transforms the summary into multiple semantically focused retrieval documents, such as overview, topic, outline, segment, action, entity, and fallback miscellaneous documents. Each of these derived documents is tagged with summary-specific metadata and embedded into the same retrieval infrastructure used for transcript chunks.

This design solves two problems simultaneously. First, it avoids representing the entire summary as one monolithic vector, which would be too coarse for topic-level matching. Second, it allows broad or transcript-level queries to retrieve focused summary sections instead of only low-level transcript snippets. These summary documents therefore function as retrieval artefacts, not merely display data.

```
{
  "participants": ["Host", "Guest"],
  "conversation_type": "podcast",
  "purpose": "Discuss current tech news and product updates",
  "overview": "2-4 sentence summary of the entire transcript",
  "key_topics": ["Topic A", "Topic B", "Topic C"],
  "main_topics_detailed": [
    {"title": "Android features", "summary": "The hosts review several new Android updates"}
  ],
  "participants_with_roles": [
    {"name": "Marquez", "role": "host"}
  ],
  "outline": [
    {"title": "Section title", "summary": "What was discussed"}
  ],
  "notable_segments": [
    {"title": "Trivia segment", "summary": "The hosts pivot into a light quiz section"}
  ],
  "action_items": [
    {"owner": "Alice", "task": "Send report", "deadline": "Friday"}
  ],
  "entities": [
    {"name": "Android", "type": "technology"}
  ],
}
```

Figure 4.1: Example canonical summary JSON used as the source structure for summary embedding document generation

4.4 RAG Pipeline Implementation

The RAG pipeline remains the primary subsystem for conversational question answering. Its role is to transform a user question into a grounded answer by retrieving relevant evidence, assembling context, and invoking the LLM under explicit grounding constraints. [4]

4.4.1 Transcript Chunking and Shared Indexing

At ingestion time, transcripts are parsed from SRT into structured segments, enriched with timestamps and speaker metadata, and merged into retrieval-sized chunks. These chunks are embedded and stored in the same global retrieval layer described earlier in the chapter, which allows the QA subsystem to reuse shared metadata filtering, transcript scoping, and summary-derived retrieval artefacts rather than maintaining a separate storage path. This shared representation was important in the final system because it kept conversational retrieval aligned with the same underlying corpus used by summarization and keyword search. Detailed implementation mechanics for transcript parsing, token-based chunk merging, document construction, and vector-store persistence are provided in Appendix A.

4.4.2 Hybrid Retrieval and Ranking

Complementary Retrieval Signals

Retrieval itself is implemented as a hybrid process rather than a single semantic search call. As illustrated in Figure 4.2, dense vector retrieval provides robustness to paraphrasing and conceptual similarity, BM25 preserves exact lexical matching for names and technical terms, phonetic retrieval improves recall when ASR introduces sound-alike spelling distortions, and fuzzy matching captures minor character-level corruption in rare tokens. [7, 16, 33, 40, 41] These signals are intentionally complementary because transcript corpora contain both semantic variation and transcription noise, making any one method insufficient across all query types.

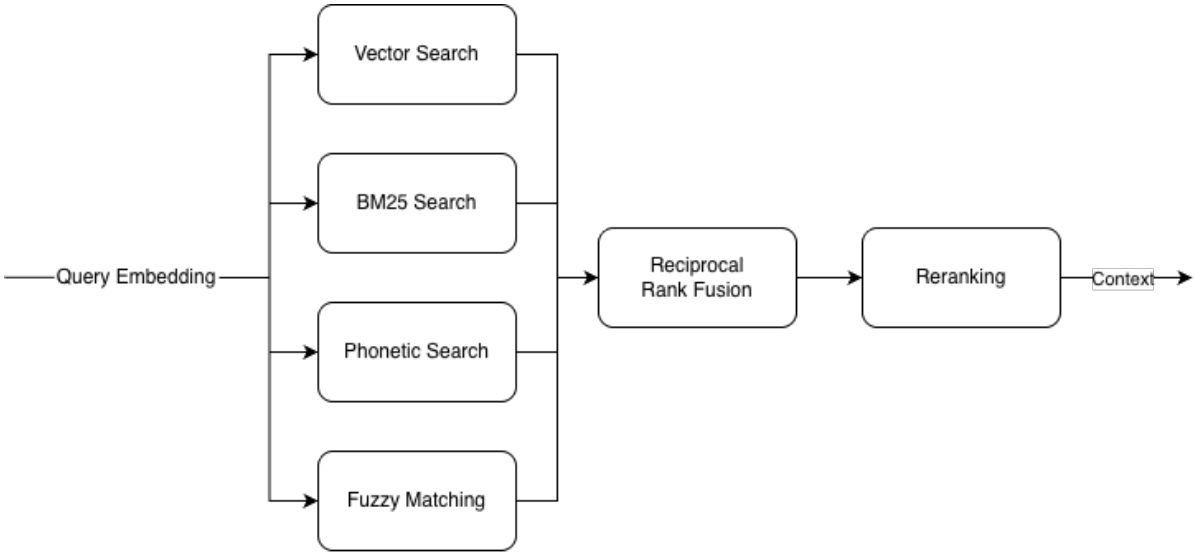

Figure 4.2: Hybrid retrieval, fusion, and reranking in the RAG subsystem

Role of Individual Retrieval Methods

Each retrieval signal addresses a different failure mode. Vector retrieval is strongest when the user paraphrases content that does not appear verbatim in the transcript. BM25 is most reliable when the query contains explicit lexical anchors such as product names, brands, or technical terms. Phonetic retrieval is valuable when automatic speech recognition preserves pronunciation but distorts spelling, while fuzzy matching helps recover rare terms affected by small character-level corruption. In combination, these methods produce a broader and more robust candidate set than any single retriever could provide on its own.

Fusion and Precision-Oriented Reranking

The outputs of these retrievers are not passed directly to generation. Instead, the system first combines them with Reciprocal Rank Fusion to reward candidates that are consis-

tently relevant across methods, and then applies a cross-encoder reranker to the fused candidate pool for precision-oriented final ordering. [10, 11] This two-stage ranking strategy balances recall and precision: the first stage casts a broad net over noisy transcript data, while the second sharpens the evidence set that is ultimately shown to the language model. The detailed implementation of the individual retrievers, rank fusion, and reranker response handling is deferred to Appendix A.

4.4.3 Context Assembly and Prompt Construction

After reranking, the selected chunks are formatted into grounded context blocks that preserve transcript identifiers, timestamps, and citation metadata. This context is then assembled into a constrained prompt together with the user's question and chat history, so answer generation remains explicitly tied to retrievable evidence rather than free-form recall. For broader or more ambiguous questions, the pipeline can also inject summary-derived framing so that transcript-level context and fine-grained evidence work together within the same prompt construction flow. [29, 42, 43]

As a final engineering design choice, the RAG pipeline was kept modular rather than collapsed into one opaque end-to-end model call. Chunk construction, hybrid retrieval, reranking, and prompt assembly can therefore be tuned independently as transcript characteristics or product requirements change. Detailed implementation mechanics, data structures, and supporting screenshots are deferred to Appendix A.

4.5 Keyword-Based Search Implementation

The keyword-based search subsystem complements the chatbot by prioritising direct retrieval rather than answer generation. It is intended for users who want inspectable search results quickly, such as all mentions of a product, statements by a specific speaker, or snippets within a particular date range.

4.5.1 Keyword-First Retrieval Design

The keyword-based search subsystem reuses the same retrieval primitives as the RAG pipeline, namely vector search, BM25, phonetic search, fuzzy matching, Reciprocal Rank Fusion, and reranking. Its orchestration, however, is designed for global search across the transcript collection rather than transcript-scoped conversational QA. In this interaction mode, BM25 is weighted more heavily so that exact lexical matches dominate for queries involving names, brands, technical terms, and other explicit keywords.

This design also changes the presentation granularity of the results. The chatbot operates within a selected transcript and retrieves a flat set of evidence chunks to support answer generation, whereas the search bar performs global retrieval and returns results grouped by transcript. Grouping at transcript level is more appropriate for search-oriented in-

teraction because it allows users to identify the most relevant transcripts first before examining the supporting snippets within each result group.

4.5.2 Two-Phase Search Flow

The search subsystem performs retrieval in two phases. In Phase 1, the system ranks transcripts using summary-derived documents only. This provides a transcript-level relevance signal that is cleaner than mixing all low-level content chunks together. Summary chunks are especially useful here because they represent the high-level contents of a transcript and can therefore act as transcript proxies.

In Phase 2, the selected transcripts are searched again at chunk level. Retrieval is scoped to one transcript at a time, the fused results are reranked, and the final top chunks are returned as grouped results.

Figure 4.3: Keyword-based search workflow in the final system

4.5.3 Date and Speaker Filtering

The search bar supports natural-language date filters and speaker filters in addition to free-text queries. Date expressions are parsed into time ranges using the dateparser library, then matched against transcript upload metadata to ensure all relevant transcripts are considered before chunk-level retrieval. [44] Speaker filtering relies on speaker metadata extracted during transcript parsing and mapped onto the corresponding chunk metadata, enabling transcript-scoped filtering by one or more speakers.

4.5.4 Relationship to the RAG Subsystem

Although the search bar and chatbot share the same retrieval building blocks, they are optimised for different interaction styles. The RAG subsystem uses retrieval to assemble grounded context for answer generation, whereas the search subsystem uses retrieval to return directly inspectable snippet lists. This separation avoids overloading the chatbot interface with direct lookup tasks while still allowing both subsystems to benefit from the same shared indices and metadata.

4.6 Deployment Implementation

The final step in the project was to expose these capabilities through a deployable retrieval server so that the implemented pipelines could be used by the broader Meadow9+ system rather than remaining as development-only components. The core deployment architecture has already been implemented, and most operational interfaces required by the application are already in place. Although the system has not yet been rolled out as a full production deployment, moving from the current implementation to production mainly requires environment-specific integration and infrastructure changes rather than a redesign of the retrieval system itself.

4.6.1 Retrieval Server Deployment Architecture

The implemented retrieval system is packaged as a separate FastAPI-based retrieval server, consistent with the architectural direction introduced in Chapter 3. This deployment decision keeps the new retrieval logic isolated from the existing Meadow9+ backend while still making the system accessible through a stable service interface. In practice, the runtime is composed of four cooperating services: a FastAPI application layer, a generation service for summarization and answer synthesis, an embedding service for dense retrieval, and a reranking service for precision-oriented evidence ordering. This separation preserves the modular implementation described earlier in the chapter while making the runtime easier to maintain, tune, and extend. [31, 32, 36]

The service-oriented deployment also reflects the role-specific model design introduced in Section 4.2.4. Embedding, reranking, and generation are not treated as one monolithic inference endpoint; instead, each is exposed as an independent runtime component. This allows individual services to be replaced, reconfigured, or scaled without redesigning the rest of the retrieval pipeline, which is important for a system expected to evolve as better local models become available. In this sense, the current retrieval server should be understood as an integration-ready service layer for Meadow9+ rather than as a claim that the final production rollout has already been completed.

4.6.2 Persistent Runtime Storage

The deployed server uses persistent storage with a clear separation of responsibilities. Structured transcript metadata, raw transcript text, upload timestamps, and canonical summary outputs are currently stored in SQLite so that uploaded transcripts remain accessible across server restarts and can be listed, retrieved, and filtered through the application layer. In the present implementation, SQLite functions primarily as a lightweight placeholder for the MongoDB-based transcript storage already used by the existing Meadow9+ server, allowing the retrieval workflow to be implemented and tested locally without depending on direct production-database integration. In parallel, transcript chunks and summary-derived retrieval documents are embedded and stored in a persistent Chroma vector store, allowing the same deployed corpus to support semantic retrieval, summary-level transcript ranking, and transcript-scoped chunk search. [37, 38]

This storage design is important because it avoids treating retrieval as an in-memory prototype while still remaining compatible with the broader Meadow9+ architecture. Since SQLite is only simulating the existing MongoDB-backed transcript store, integration into the production environment mainly requires adapting the persistence layer to the server's existing database workflow, while leaving the downstream retrieval pipeline unchanged.

4.6.3 End-to-End Deployed Processing Flow

When a new transcript is uploaded, the deployed server validates the input, parses the transcript into structured segments, constructs retrieval-sized chunks, generates the transcript summary, stores transcript artefacts in persistent storage, and then indexes both chunk documents and summary-derived documents into the shared vector store. As a result, ingestion, summarization, persistence, and indexing are all handled within one deployed runtime path rather than through disconnected development scripts.

In the current implementation, this workflow is triggered via transcript-upload endpoints, enabling the full pipeline to be tested and evaluated independently during development. For production deployment, the same ingestion pipeline can be seamlessly adapted to receive transcript data directly from the Meadow9+ backend server instead of manual uploads. Under this integration, only the source of the transcript data changes, while all downstream processing stages—namely parsing, chunking, summarization, persistence, indexing, and retrieval—remain unchanged.

Once transcripts have been ingested, the same deployment supports the system's three user-facing interaction modes. The backend can list uploaded transcripts, return stored summary or raw-content views for a selected transcript, execute keyword-based global search over the uploaded corpus, and serve transcript-aware chatbot or one-off query requests through the RAG pipeline. This end-to-end deployment is therefore not limited to inference alone; it operationalizes the entire transcript lifecycle from upload to retrieval-driven interaction.

4.6.4 Streaming Chat Responses

The deployed retrieval server also supports streaming chat responses, which was missing from the Meadow9+ chatbot implementation. Instead of waiting for the full answer to be generated before returning any output, the server can stream partial answer tokens to the client as they are produced. This improves perceived latency because users receive visible progress early rather than experiencing one long blocking delay.

4.6.5 Retrieved Evidence in Chat Responses

The deployed chat interface also returns the retrieved transcript chunks used to support answer generation. Rather than exposing only a final generated response, the system can surface supporting transcript excerpts together with metadata such as timestamps

and source identifiers. This design directly reflects the transparency goals discussed in Chapter 2, where Meadow9+ was motivated as a system that should expose inspectable evidence rather than behave as a fully opaque chatbot.

Returning retrieved chunks improves verifiability because users can inspect which transcript segments were actually used to ground an answer. This in turn improves user trust and confidence, especially in professional settings where incorrect attribution or hallucinated conclusions may have practical consequences.

4.6.6 Operational Interfaces and Health Monitoring

The FastAPI layer provides the operational boundary between the retrieval server and the rest of the system. Instead of exposing internal LangChain pipelines directly, the implementation wraps transcript ingestion, transcript retrieval, search, question-answering, streaming chat, and health-monitoring capabilities behind explicit service interfaces. This makes the deployed system easier to integrate with the Meadow9+ backend and keeps application-facing behavior separate from the lower-level implementation details of chunking, retrieval, and prompting. [31, 32] A compact summary of the implemented interface categories is provided in Appendix A, Table A.1.

Operational monitoring is also built into the deployment. A health-check interface reports whether the retrieval runtime and its upstream dependencies are available, including the embedding, reranking, and generation services, as well as whether the retrieval layer has been initialized successfully. This is a practical deployment requirement because the final system depends on multiple cooperating components rather than one standalone process. Detecting service availability at runtime makes failures easier to diagnose and provides a clearer integration contract for the surrounding application stack.

Overall, the implemented deployment transforms the Hybrid Retrieval System from a set of individual pipelines into a usable backend service. The final system is therefore not only a prototype of summarization, RAG, and search techniques, but an operational retrieval server implementation whose core interfaces and processing paths are already in place, making integration into Meadow9+ achievable with relatively small deployment-specific changes rather than a fundamental redesign.

4.7 Prototype Flutter Client

In addition to the backend retrieval server, a prototype Flutter client was also implemented to exercise the new retrieval capabilities through a mobile-oriented user interface. This frontend work is not a primary contribution of the project and remains technically outside the formal scope of the retrieval-system implementation. It is nevertheless useful to mention briefly because it demonstrates that the deployed backend interfaces can already support an end-to-end application workflow.

The prototype client exposes the three main user-facing interaction modes introduced in

this project. The summary view presents the generated meeting summary in a structured form rather than as one block of plain text, making the output easier to scan and navigate. The Ask AI view surfaces the number of retrieved sources used to answer a question and allows users to expand those supporting chunks, directly operationalizing the transparency and confidence goals discussed earlier in this chapter. The global search view integrates the keyword-based transcript search workflow, showing that the same backend can also support direct retrieval-oriented interaction outside the chatbot interface.

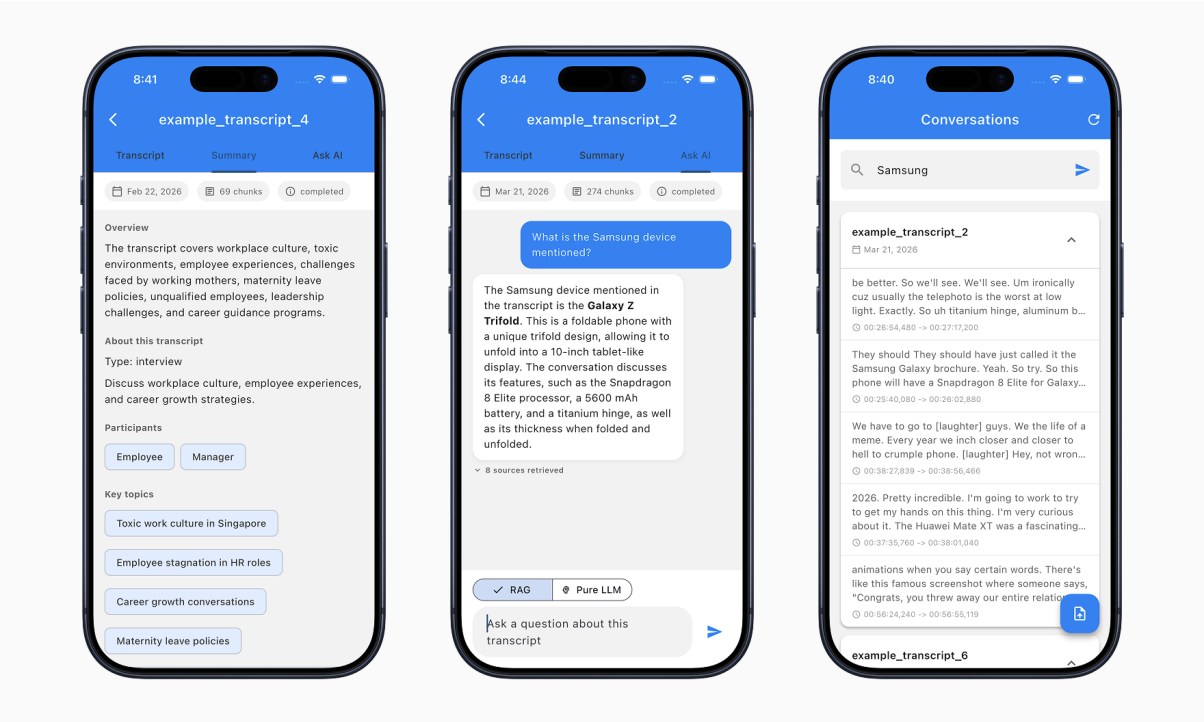

Figure 4.4: Prototype Flutter client screenshots showing the structured summary tab, Ask AI tab with expandable retrieved sources, and global keyword search interface

Chapter 5

Evaluation and Results

This chapter presents the evaluation methodology, experimental results, and discussion used to assess the RAG pipeline subsystem and its end-to-end performance. The comparative evaluation centers on the RAG pipeline as this is the subsystem that can be most directly assessed against the existing chatbot in terms of performance, scalability, and latency.

5.1 Evaluation Pipeline Implementation

The evaluation pipeline was designed to assess both retrieval quality and end-to-end answer quality while remaining lightweight enough to support repeated experimentation during system development. Detailed prompt templates, schemas, implementation screenshots, and extended worked examples are provided in Appendix B and Appendix C.

5.1.1 Dataset Generation

The evaluation dataset was generated automatically using an LLM-based process rather than fully manual annotation. Manual construction of question-answer benchmarks across long meeting transcripts would have been slow, costly, and difficult to scale, especially when the retrieval pipeline was still being tuned [45]. Recent work has shown that LLM-assisted evaluation can substantially reduce this overhead while still supporting systematic comparison [46].

To create the dataset, each transcript was first grouped into larger context windows and then passed to an LLM to synthesize grounded question-answer pairs. Each QA item retained the generated question and answer together with the source start timestamp, source end timestamp, and originating context text. The timestamp metadata was particularly important because it allowed retrieval relevance to be checked algorithmically through temporal overlap rather than manual judging. Appendix B documents the grouping example, generation prompt, API-chain setup, and ground-truth JSON structure used in this process.

Importantly, the generated dataset included a mixture of narrow fact-extraction questions and higher-level, less localized questions, such as asking for a summary of the meeting. This broader question distribution was intentional to reduce bias toward RAG-style retrieval evaluation, since a benchmark made up only of highly specific span-level questions would naturally favor systems optimized for retrieving small, precise evidence windows. Including both question types therefore provides a more balanced assessment of how the

system performs across realistic transcript-query patterns.

5.1.2 Evaluation Metrics

The pipeline reports two retrieval metrics and two generation metrics so that failures can be attributed to the correct stage of the RAG system.

Hit Rate measures the proportion of questions for which at least one retrieved chunk overlaps the ground-truth timestamp range [27]. It is used as a recall-oriented indicator of whether the retriever surfaced any relevant evidence at all.

Mean Reciprocal Rank (MRR) measures how highly the first relevant chunk is ranked [9]. It complements Hit Rate by distinguishing between systems that eventually retrieve relevant evidence and systems that place that evidence near the top of the result list.

F1 Score measures token-level overlap between the generated answer and the reference answer [27]. It captures lexical fidelity, but it may undervalue correct paraphrases or grounded elaborations.

LLM-Judge Accuracy uses a separate LLM to assess whether the generated answer is semantically consistent with the reference answer [47]. This metric captures semantic correctness more directly than F1, though it remains sensitive to the design of the judge prompt and the specificity of the ground truth.

Retrieval and generation were evaluated separately for diagnostic clarity. Hit Rate and MRR are computed before answer generation and therefore isolate retrieval behavior, while F1 and LLM-judge accuracy are computed after generation and therefore reflect how well the language model used the retrieved evidence. This separation made it possible to distinguish retrieval failures from answer-generation failures during experimentation. Appendix B contains the temporal-overlap logic, F1 preprocessing details, and judge prompt used in the implementation, while Appendix C contains extended diagnostic examples.

5.1.3 Speed Benchmarking Design

Latency was evaluated using a separate speed benchmarking pipeline so that runtime comparisons between the RAG system and a Pure LLM baseline would remain fair. Both methods were tested on the same question set derived from the evaluation dataset, ensuring that quality and latency analyses were based on comparable workloads.

The benchmark used interleaved execution rather than evaluating one system fully before the other. This design reduced bias from prompt caching in modern LLM services, which could otherwise make later runs appear artificially faster [48]. A short warmup phase was also applied before timed runs to reduce cold-start effects from model loading, runtime initialization, and connection setup, with warmup executions excluded from the final statistics. Appendix B provides the interleaving illustration together with the supporting

latency-statistics and comparative-metrics tables.

5.2 Experimental Results

This section presents the empirical results from evaluating the RAG system across multiple transcripts, covering both quality metrics (retrieval and generation performance) and speed performance (comparison with the Pure LLM baseline).

The reported experiments were conducted using a locally hosted development LLM rather than the self-hosted model deployed on the Meadow9+ production server. As a result, the absolute latency and generation quality reported here should be interpreted as development-stage measurements rather than final production benchmarks.

5.2.1 RAG Quality Metrics

The RAG quality evaluation indicates that the proposed retrieval pipeline performs reliably across most of the evaluated question set. In general, the system was able to retrieve the correct supporting evidence for the majority of queries, and end-to-end answer quality was also reasonably strong given the difficulty of evaluating grounded natural-language answers. Taken together, these results suggest that the core retrieval design is effective and that the main remaining limitations lie less in evidence discovery and more in downstream answer generation and evaluation strictness.

Table 5.1 presents the per-transcript RAG quality metrics.

Transcript	**Duration**	**Questions**	**Hit Rate**	**MRR**	**Accuracy**	**F1 Score**
Transcript 1	00:08:21	29	1.000	1.000	1.000	0.677
Transcript 2	01:59:40	150	0.867	0.867	0.767	0.476
Transcript 3	00:08:14	32	1.000	1.000	0.875	0.613
Transcript 4	00:36:14	62	1.000	1.000	0.800	0.446
Transcript 5	00:34:08	59	1.000	1.000	0.900	0.512
Transcript 6	00:36:14	58	1.000	1.000	0.733	0.478
Aggregate	**04:02:51**	**390**	**0.949**	**0.949**	**0.775**	**0.503**

Table 5.1: Per-transcript RAG quality metrics aggregated results

Aggregate Interpretation

Retrieval performance. Under the six-transcript evaluation set, the system achieved a question-weighted Hit Rate of 0.949 and an MRR of 0.949. This indicates that relevant evidence was retrieved and ranked highly for most questions, although Transcript 2 remains the main source of retrieval misses.

Generation performance. Using the per-transcript values, the system achieved a

question-weighted aggregate accuracy of 77.5% and an average F1 score of 0.503. This remains consistent with the earlier observation that retrieval is generally strong, while end-to-end answer quality is more sensitive to generation behavior and judge strictness.

False Negatives from Semantic Judge

One recurring evaluation issue was false negatives from the semantic judge. In some cases, the generated answer was more specific than the reference answer while still remaining grounded in the transcript, causing the judge to mark it as incorrect despite the answer being substantively valid. To reduce the impact of such cases during analysis, contradictory judgments between the RAG and Pure LLM outputs were reviewed more carefully through a secondary manual-assisted process: the transcript together with both candidate answers was provided to an LLM for closer comparison, allowing obviously grounded but more specific answers to be identified before interpretation of the final results. A fuller example and additional diagnostic cases are provided in Appendix C.

Overall, the quality evaluation supports the conclusion that the RAG subsystem is technically sound for transcript-grounded question answering. Retrieval performance is consistently high, and answer accuracy is generally good despite the inherent difficulty of judging paraphrased or more specific responses.

5.2.2 Speed Performance

This is the more operationally important part of the evaluation, because it directly measures improvement over the existing Meadow9+ chatbot. While the quality evaluation shows whether the proposed RAG pipeline can answer questions correctly, the speed evaluation determines whether the new architecture delivers a practical product advantage over the current solution in terms of responsiveness and scalability under real transcript lengths.

Table 5.2 presents the per-transcript speed comparison.

Transcript	Duration	RAG Accuracy	Pure LLM Accuracy	Speedup Factor
Transcript 1	00:08:21	100.00%	100.00%	0.58x
Transcript 2	01:59:40	76.67%	53.33%	8.35x
Transcript 3	00:08:14	87.50%	100.00%	0.63x
Transcript 4	00:36:14	80.00%	70.00%	2.21x
Transcript 5	00:34:08	90.00%	66.67%	2.25x
Transcript 6	00:36:14	73.33%	70.00%	2.20x

Table 5.2: Per-transcript speed comparison showing speedup factors and end-to-end accuracy

Speedup vs Transcript Duration

The speed results show a clear length-dependent pattern. On the longer transcripts, Transcript 2, Transcript 4, Transcript 5 and Transcript 6, RAG was consistently faster than the Pure LLM baseline, with speedups ranging from 2.20x to 8.35x, and also achieved equal or better accuracy. The largest gain occurred on Transcript 2, the longest transcript, where RAG was both substantially faster and more accurate, which supports the claim that retrieval becomes increasingly beneficial as transcript length grows.

On shorter transcripts, Transcript 1 (00:08:21) and Transcript 3 (00:08:14), however, RAG was slower than the Pure LLM baseline because retrieval overhead outweighed the benefit of reducing prompt length. In these sub-10-minute cases, Pure LLM also matched or exceeded RAG in accuracy, indicating that retrieval is not always the most efficient choice for shorter inputs.

This suggests a practical direction for future work: the system could estimate transcript size or token count and switch between a Pure LLM path and a RAG-based path based on a tuned threshold. Such a threshold could be optimized empirically so that shorter transcripts avoid unnecessary retrieval overhead while longer transcripts continue to benefit from retrieval-based context reduction.

Overall, the speed evaluation provides the clearest evidence of practical improvement over the existing Meadow9+ chatbot approach. The results show that RAG is not universally better for every transcript length, but it offers a strong advantage on the longer transcripts that are most likely to stress a pure-LLM pipeline in production. This supports the view that the proposed system is not only viable as a retrieval-augmented chatbot architecture, but also meaningfully more scalable than the current baseline when transcript size grows.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This project set out to address a key limitation in the pre-project Meadow9+ chatbot: the system handled user questions by passing full meeting transcripts directly to a large language model, which led to high latency, increased computational cost, and degraded answer quality on long transcripts. Rather than treating this as a narrow chatbot-prompting problem, the project approached it as a broader transcript retrieval problem and designed a Hybrid Retrieval System that supports multiple interaction modes over the same transcript corpus.

The final system comprises three coordinated capabilities built on a shared retrieval foundation. First, a hierarchical summarization pipeline was implemented to generate high-level transcript understanding and summary-derived retrieval artefacts. Second, a RAG pipeline was implemented to support grounded conversational question answering through hybrid retrieval, reranking, and evidence-constrained prompt construction. Third, a keyword-first search workflow was implemented to support direct date-based, speaker-based, and lexical retrieval over the transcript collection. These capabilities were integrated through a shared processing backbone and exposed through a deployable retrieval server, allowing the project to move beyond isolated prototypes toward an integration-ready backend service for Meadow9+.

The evaluation results show that the proposed retrieval-driven design is effective. The RAG subsystem achieved strong retrieval quality across the evaluation set, and end-to-end answer quality was sufficiently reliable to support practical transcript-grounded question answering. In addition, the speed benchmarking results show that retrieval augmentation provides substantial latency benefits on longer transcripts, where the original full-context approach becomes increasingly inefficient.

Beyond improvements in retrieval quality and latency, the implemented system enhances the overall transcript retrieval and interaction experience by enabling streamed LLM responses and improving transparency. Specifically, the system is able to return retrieved supporting evidence alongside generated answers, rather than presenting only opaque responses as in the original chatbot design.

Overall, this project demonstrates that retrieval-driven system design provides a practical and scalable improvement over the original Meadow9+ full-context chatbot, particularly for long-duration meeting transcripts. By approaching the retrieval problem from multiple complementary angles, the system extends Meadow9+ beyond a transcription tool with limited post-hoc interaction into a more capable platform for transcript understanding and retrieval. Although further evaluation and deployment work remain, the architectural design, implementation, and empirical results presented in this report

establish the technical viability of the proposed hybrid retrieval approach.

6.2 Future Work

Although the core Hybrid Retrieval System has been implemented and evaluated, several important areas remain for future work.

6.2.1 Adaptive Routing for Transcript Length

The final system could benefit from adaptive routing between a pure-LLM path and a retrieval-augmented path. As shown in Chapter 5, retrieval provides a clear advantage on longer transcripts, but its overhead is not always beneficial for very short transcripts. A practical next step is therefore to introduce a routing policy based on transcript length or token count, so that shorter transcripts can bypass unnecessary retrieval stages while longer transcripts continue to benefit from retrieval-based context reduction. This would make the deployed system more efficient across a wider range of transcript lengths.

6.2.2 Multiple-Language Support

The current implementation was designed with multilingual extension in mind. In particular, the selected LLM, embedding model, and reranker were chosen so that answer generation, dense retrieval, and reranking can in principle support multiple languages rather than only English. However, the evaluation presented in this report was conducted only on English-language transcripts, and no explicit benchmarking has yet been performed on transcripts in other languages. Future work should therefore assess how well the hybrid retrieval pipeline generalizes to multilingual meeting data, including both retrieval quality and end-to-end answer quality under non-English transcript conditions.

6.2.3 Production Hardening and Integration

Additional work is needed to harden the retrieval server for production use within the broader Meadow9+ environment. Although the current implementation already provides a deployable service architecture, full deployment still requires tighter integration with the existing Meadow9+ backend and persistence workflow, together with stronger operational monitoring, scalability planning, and service robustness under real production workloads. These tasks are primarily deployment and infrastructure work rather than changes to the core retrieval design, but they are necessary for complete operational adoption.

6.2.4 Refinement of Answer Evaluation

The answer-evaluation methodology can be further improved. As discussed in Chapter 5 and Appendix C, the current semantic judging setup can produce false negatives when generated answers are more specific than the reference answer while still remaining grounded in the transcript. Future work can refine the judging framework to better distinguish genuinely incorrect responses from grounded elaborations, thereby producing a more faithful measure of end-to-end answer quality.

Appendix A

Detailed RAG Implementation

This appendix collects the lower-level implementation details, intermediate data representations, and supporting screenshots for the RAG subsystem described at a higher level in Chapter 4. It is intended as supporting engineering documentation rather than the main narrative of the final report.

A.1 Implemented Retrieval Server Interfaces

Table A.1 provides a compact summary of the main retrieval server interfaces implemented for integration with Meadow9+. The table is intended to demonstrate implementation coverage at a high level without reproducing the full API contract in the body chapters.

Method	Interface Category	Purpose
GET	Service metadata	Expose basic service metadata, documentation entry points, and service discovery information for integration.
GET	Health monitoring	Report runtime readiness, retrieval initialization state, and dependency availability so failures can be detected operationally.
POST	Transcript ingestion	Accept uploaded transcript input for validation, parsing, chunking, summarization, persistence, and indexing into the retrieval layer.
GET	Transcript listing and retrieval	List available transcripts and return stored transcript summaries or raw transcript content for application use.
POST	Keyword-based search	Execute corpus-level transcript search and return grouped transcript-level matches with supporting chunks.
POST	One-off query	Perform stateless grounded question answering over the uploaded transcript corpus.
POST	Multi-turn chat	Support transcript-aware conversational question answering with retrieved evidence returned alongside the generated answer.
POST	Streaming chat	Stream partial chat responses to reduce perceived latency while returning final retrieval metadata at completion.
GET / DELETE	Session management	List active chat sessions or delete a session when conversation state should be cleared.

Table A.1: High-level summary of implemented retrieval server interfaces

A.2 Transcript Parsing, Chunking, and Storage

The RAG ingestion path begins by parsing SRT transcripts into structured records containing text, timestamps, sequence numbers, and speaker metadata. Speaker detection handles both simple markers and explicit speaker-name prefixes so that speaker information can later be used for filtering and citation without polluting the searchable transcript text.

```
1
00:00:00,640 --> 00:00:15,140
Hi, how are you today?

2
00:00:15,140 --> 00:01:31,240
>> Speaker: Read this aloud for me...
```

Figure A.1: SRT input format with sequence numbers and timestamps

```
{
    'sequence': 1,
    'start_time': '00:00:00,640',
    'end_time': '00:00:15,140',
    'text': 'Hi, how are you today?',
    'speaker': None  # or 'Speaker' if ">>" prefix detected
}
```

Figure A.2: Parsed SRT structure with speaker metadata extraction

After parsing, small subtitle units are merged into retrieval-sized chunks using token-counted boundaries. This is more reliable than character-based estimation because it aligns chunk sizing with the practical limits of the embedding and generation models, while preserving natural subtitle boundaries for later timestamp attribution and answer grounding.

```
1. For each chunk:
   a. Count tokens using TokenCounter
   b. If chunk < MIN_CHUNK_SIZE (25 tokens):
      - Add to accumulator
      - If accumulator >= TARGET_MERGED_SIZE (128 tokens):
        * Create merged chunk
        * Update timestamps (earliest start, latest end)
        * Track merged_count
   c. Else:
      - Flush accumulator if non-empty
      - Emit chunk as-is

2. Flush remaining accumulator
```

Figure A.3: Token-based merging algorithm for chunk size optimization

The merged chunks are then converted into LangChain ‘Document‘ objects with transcript identifiers, timestamps, chunk IDs, and speaker information attached as metadata. This representation is used consistently across retrieval, reranking, filtering, and citation formatting.

```python
def process_transcript(self, transcript_path: str) -> List[Document]:

    filename = os.path.basename(transcript_path)
    print(f"Processing {filename}...")

    # Parse SRT file into chunks
    chunks = parse_srt_chunks(transcript_path)
    print(f"  Parsed {len(chunks)} SRT entries")

    # Create documents (with merging if enabled)
    documents = create_documents_from_srt_chunks(
        chunks,
        file_path=filename
    )

    # Add transcript_id metadata (cleaned filename without extension)
    transcript_id = filename.replace('.srt', '').replace('.', '_')
    for doc in documents:
        doc.metadata['transcript_id'] = transcript_id

    print(f"  Created {len(documents)} document chunks")
    return documents
```

Figure A.4: Document processing pipeline

Embedding generation is one of the most computationally expensive stages in indexing, so the implementation relies on batched vector-store integration rather than one document at a time. Chroma persists the resulting embeddings and metadata in local storage, while incremental addition supports appending newly uploaded transcripts without rebuilding the entire index. [37, 38]

```python
# Create vector store
print(f"  Creating Chroma vector store (this will take a while)...")

vector_store = Chroma.from_documents(
    documents=all_documents,
    embedding=self.embeddings,
    collection_name=self.collection_name,
    persist_directory=self.global_store_path
)

print("  ✓ Vector store created and persisted")
```

Figure A.5: Vector store creation from documents

```python
print(f"\n  Adding {len(all_new_documents)} new chunks to vector store...")

# Pass explicit IDs to enable upsert behavior (prevent duplicates)
ids = [doc.id for doc in all_new_documents if hasattr(doc, 'id') and doc.id]
if ids and len(ids) == len(all_new_documents):
    vector_store.add_documents(all_new_documents, ids=ids)
else:
    vector_store.add_documents(all_new_documents)

print("  ✓ New documents added to vector store")
```

Figure A.6: Incremental document addition

```
vector_stores/global_combined/
├── chroma.sqlite3              # Main vector database (embeddings + metadata)
├── phonetic_index.db           # Phonetic search index (SQLite)
└── bm25_index/
    └── bm25_index.pkl          # Serialized BM25 index
```

Figure A.7: ChromaDB storage directory structure

A.3 Hybrid Retrieval Internals and Reranking

Stage 1 retrieval combines four complementary signals: dense vector search for semantic similarity, BM25 for exact lexical evidence, phonetic search for ASR-induced spelling distortions, and fuzzy matching for character-level corruption in rare tokens. [16,33,40,41]

Vector search operates over dense embeddings stored in Chroma and supports optional metadata filtering for transcript-scoped retrieval.

```python
def run_semantic_search(
    vector_retriever,
    query: str,
    k: int,
    chroma_filter: Optional[Dict] = None
) -> List[Document]:

    if chroma_filter:
        return vector_retriever.vectorstore.similarity_search(
            query, k=k, filter=chroma_filter
        )
    else:
        return vector_retriever.invoke(query)
```

Figure A.8: Vector search core function showing semantic retrieval with optional metadata filtering

BM25 provides exact keyword matching and is especially useful for proper nouns, brands, acronyms, and technical terms that should appear verbatim in the retrieved evidence.

```python
@staticmethod
def _default_tokenizer(text: str) -> List[str]:

    return text.lower().split()
```

Figure A.9: BM25 tokenization function using lowercase split

```python
def __init__(
    self,
    documents: List[Document],
    tokenizer: Optional[Callable[[str], List[str]]] = None
):

    if not documents:
        raise ValueError("Cannot create BM25 retriever with empty document list")

    self.documents = documents
    self.tokenizer = tokenizer or self._default_tokenizer

    # Tokenize all documents for BM25 indexing
    print(f"Tokenizing {len(documents)} documents for BM25 indexing...")
    tokenized_corpus = [
        self.tokenizer(doc.page_content)
        for doc in documents
    ]

    # Build BM25 index
    print("Building BM25 index...")
    self.bm25 = BM25Okapi(tokenized_corpus)
    print(f"✓ BM25 index created with {len(documents)} documents")
```

Figure A.10: BM25 index initialization with corpus tokenization

```python
# Tokenize query
tokenized_query = self.tokenizer(query)

if filter_value:
    # Normalize filter_value to list for consistent handling
    if isinstance(filter_value, str):
        filter_values = [filter_value]
    else:
        filter_values = filter_value

    # Filter documents by specified field before ranking
    filtered_indices = [
        i for i, doc in enumerate(self.documents)
        if doc.metadata.get(filter_field) in filter_values
    ]

    if not filtered_indices:
        # No documents match the filter
        return []

    # Get BM25 scores efficiently for only the subset of documents
    # get_batch_scores returns a list of scores corresponding to doc_ids
    scores = self.bm25.get_batch_scores(tokenized_query, filtered_indices)

    # Zip indices and scores together
    filtered_scores = list(zip(filtered_indices, scores))
```

Figure A.11: BM25 query tokenization and metadata filtering

Phonetic search uses Double Metaphone to recover recall when ASR output preserves pronunciation but not spelling, especially for names and rare technical vocabulary. [40,49]

```python
def tokenize_and_encode(self, text: str) -> List[Tuple[str, str]]:

    # Lowercase and extract words (alphabetic only)
    words = re.findall(r'[a-zA-Z]+', text.lower())

    # Filter short words (< 3 chars) - they're too common for phonetic matching
    words = [w for w in words if len(w) >= 3]
```

Figure A.12: Phonetic word tokenization and filtering

```python
    # Compute phonetic codes
    phonetic_pairs = []
    seen = set()

    for word in words:
        # Get primary phonetic code (ignore alternate)
        code, _ = doublemetaphone(word)

        if code and (code, word) not in seen:
            phonetic_pairs.append((code, word))
            seen.add((code, word))

    return phonetic_pairs
```

Figure A.13: Double Metaphone phonetic encoding

```
"Samsung" → "SMSNK"
"Samson" → "SMSN"
```

Figure A.14: Phonetic encoding examples for homophone matching

```python
# Build query with optional transcript filter
if transcript_filters:
    # With filter (single or multiple transcripts)
    code_placeholders = ','.join('?' * len(query_codes))
    transcript_placeholders = ','.join('?' * len(transcript_filters))
    sql = f"""
        SELECT chunk_id, COUNT(*) as overlap_count
        FROM phonetic_codes
        WHERE phonetic_code IN ({code_placeholders})
          AND transcript_id IN ({transcript_placeholders})
        GROUP BY chunk_id
        ORDER BY overlap_count DESC
    """
    params = query_codes + transcript_filters
```

Figure A.15: Phonetic retrieval SQL query with transcript filtering

```python
def run_phonetic_search(
    phonetic_indexer,
    query: str,
    vector_store,
    k: int,
    transcript_filters: Optional[List[str]] = None
) -> List[Document]:

    # Get phonetic matches from indexer
    phonetic_results = phonetic_indexer.retrieve(
        query,
        transcript_filters=transcript_filters,
        top_k=k
    )

    # Convert to scores dict
    phonetic_scores = {chunk_id: count for chunk_id, count in phonetic_results}

    # Build chroma filter for document fetching
    chroma_filter = None
    if transcript_filters:
        if len(transcript_filters) == 1:
            chroma_filter = {"transcript_id": {"$eq": transcript_filters[0]}}
        else:
            chroma_filter = {"transcript_id": {"$in": transcript_filters}}

    # Convert scores to ranked document list
    return _convert_scores_to_ranked_list(
        phonetic_scores,
        vector_store,
        chroma_filter
    )
```

Figure A.16: Phonetic score conversion and document retrieval

Fuzzy matching complements phonetic search by using edit-distance similarity over rare tokens detected with Zipf frequency. This keeps the computation targeted toward tokens most likely to contain ASR errors or spelling variation. [50, 51]

```python
def is_rare_token(self, token: str) -> bool:

    # Get Zipf frequency (log10 of occurrences per billion words)
    # 0.0 means not in dictionary (infinite rarity)
    freq = zipf_frequency(token, 'en')

    # Threshold 4.0 corresponds to 10^4 per billion (1 in 100,000)
    # Common words > 4.0, Rare words < 4.0
    return freq < 4.0
```

Figure A.17: Rare token detection using Zipf frequency

```python
def extract_rare_tokens(self, query: str) -> List[str]:

    # Tokenize (extract alphanumeric words)
    tokens = re.findall(r'\w+', query)

    # Filter to rare tokens
    rare_tokens = [t for t in tokens if self.is_rare_token(t)]

    return rare_tokens
```

Figure A.18: Extracting rare tokens from query

```python
def compute_fuzzy_scores(
    self,
    query: str,
    candidate_pool: List[Document],
    top_k: Optional[int] = None
) -> Dict[int, float]:

    # Extract rare tokens from query
    rare_tokens = self.extract_rare_tokens(query)

    if not rare_tokens:
        return {}

    # Compute scores
    scores = {}
```

Figure A.19: Fuzzy scoring algorithm: setup and initialization

```python
for doc in candidate_pool:
    chunk_id = doc.metadata.get('chunk_id')
    if chunk_id is None:
        continue

    doc_tokens = self._tokenize_document(doc)
    if not doc_tokens:
        continue

    # Compute best match score
    max_score = 0.0
    for query_token in rare_tokens:
        for doc_token in doc_tokens:
            # Compute edit-distance similarity using RapidFuzz
            score = fuzz.ratio(
                query_token.lower(),
                doc_token.lower()
            ) / 100.0  # Normalize to 0-1 scale

            max_score = max(max_score, score)

            # Early exit if we found a near-perfect match
            if max_score >= 0.95:
                break

        if max_score >= 0.95:
            break

    # Only include if above threshold
    if max_score >= self.threshold:
        scores[chunk_id] = max_score
```

Figure A.20: Fuzzy scoring algorithm: nested loop with RapidFuzz ratio

```python
# Sort by score and limit to top_k
if top_k is not None and len(scores) > top_k:
    sorted_items = sorted(
        scores.items(),
        key=lambda x: x[1],
        reverse=True
    )[:top_k]
    scores = dict(sorted_items)

return scores
```

Figure A.21: Fuzzy top-k selection and ranking

These four retrievers operate independently and are therefore executed in parallel before their results are fused. Table A.2 summarizes their respective roles, index types, and scoring signals.

Search Method	Algorithm	Index Type	Scoring Metric	Top-K	Best For
Vector Search	Cosine similarity over embeddings	Dense vectors (Chroma)	Embedding distance	10	Semantic similarity, paraphrasing
BM25 Search	Probabilistic TF-IDF ranking	Inverted index	BM25 score	25	Keyword matching, proper nouns
Phonetic Search	Double Metaphone encoding	SQLite phonetic index	Overlap count	10	ASR errors, homophones
Fuzzy Matching	Edit-distance (RapidFuzz)	No index (full scan)	Fuzzy ratio	10	Typos, spelling variations

Table A.2: Search method characteristics in the hybrid retrieval stage used by the RAG subsystem

After Stage 1 retrieval, the system applies Reciprocal Rank Fusion to normalize heterogeneous rankings and reward candidates that score well across multiple retrieval signals. [10]

```
score(d) = Σ weight_i * (1 / (k + rank_i(d)))

where:
  k = 60 (HYBRID_FUSION_K constant from literature)
  weight_i = search method weight (VECTOR_WEIGHT, BM25_WEIGHT, PHONETIC_WEIGHT, FUZZY_WEIGHT)
  rank_i(d) = rank of document d in search method i (1-indexed)

Configured weights:
  VECTOR_WEIGHT = 1.0    (primary signal)
  BM25_WEIGHT = 1.0      (primary signal)
  PHONETIC_WEIGHT = 0.2  (supplementary signal)
  FUZZY_WEIGHT = 0.2     (supplementary signal)
```

Figure A.22: Reciprocal Rank Fusion weights for the hybrid retrieval signals

The fused pool is then reranked with a cross-encoder, which jointly scores each query-document pair and provides a more precise final ordering than the initial retrieval signals alone. [11]

```
{
  "results": [
    {"index": 2, "relevance_score": 0.9523},
    {"index": 0, "relevance_score": 0.8876},
    {"index": 5, "relevance_score": 0.7234},
    {"index": 1, "relevance_score": 0.6891}
  ]
}
```

Figure A.23: Expected reranker response format

```python
# Map back to original documents
reranked_docs = []
for item in ranked_results:
    idx = item["index"]
    if 0 <= idx < len(documents):
        # Optionally add relevance score to metadata
        doc = documents[idx]
        # Create a copy with updated metadata
        reranked_doc = Document(
            page_content=doc.page_content,
            metadata={
                **doc.metadata,
                "rerank_score": item.get("relevance_score", 0)
            }
        )
        reranked_docs.append(reranked_doc)
```

Figure A.24: Mapping reranked results back to the original documents

```python
# Enforce top_n slicing (in case server ignores the parameter)
if top_n is not None:
    reranked_docs = reranked_docs[:top_n]

return reranked_docs
```

Figure A.25: Top-n selection after reranking

A.4 Context Formatting and Prompt Construction

After reranking, the selected chunks are converted into a structured context string for answer generation. Each chunk is formatted with identifiers and timestamps so that the generated answer remains auditable against the source transcript.

```python
def format_flat_chunks(documents: List[Document]) -> str:

    context_parts = []
    for i, doc in enumerate(documents, 1):
        chunk_id = doc.metadata.get('chunk_id', 'unknown')
        start_time = doc.metadata.get('start_time', '')
        end_time = doc.metadata.get('end_time', '')

        header = f"[Chunk {i} - ID: {chunk_id}"
        if start_time and end_time:
            header += f", Time: {start_time} -> {end_time}"
        header += "]"

        context_parts.append(f"{header}\n{doc.page_content}")

    return "\n\n".join(context_parts)
```

Figure A.26: Chunk formatting with individual metadata headers

```
[Chunk 1 - ID: 42, Time: 00:05:23 -> 00:05:45]
Speaker A discusses the project timeline and key milestones...

[Chunk 2 - ID: 43, Time: 00:05:45 -> 00:06:08]
Speaker B responds with concerns about resource allocation...
```

Figure A.27: Output format example

```python
def format_chunks_with_citations(documents) -> Dict[str, any]:

    context = format_retrieved_chunks(documents)

    # Flatten groups to individual chunks for source citations
    if documents and isinstance(documents[0], dict):
        # Grouped format - flatten for citations
        flat_docs = []
        for group in documents:
            flat_docs.extend(group['chunks'])
        documents = flat_docs

    # Generate citations
    sources = []
    for i, doc in enumerate(documents, 1):
        # Create a preview (first 100 characters)
        preview = doc.page_content[:100]
        if len(doc.page_content) > 100:
            preview += "..."

        citation = {
            'chunk_number': i,
            'chunk_id': doc.metadata.get('chunk_id', 'unknown'),
            'preview': preview,
            'full_text': doc.page_content,
            'source': doc.metadata.get('source', 'unknown'),
            'transcript_id': doc.metadata.get('transcript_id', 'unknown'),
            'start_time': doc.metadata.get('start_time', ''),
            'end_time': doc.metadata.get('end_time', '')
        }
        sources.append(citation)

    return {
        'context': context,
        'sources': sources
    }
```

Figure A.28: Citation formatting for user transparency and source attribution

The prompt layer combines system instructions, retrieved context, optional chat history, and the current user question. For broad or ambiguous requests, the same prompt-construction path can also inject summary-derived transcript framing before answer generation. [29, 42, 43]

```
RAG_SYSTEM_MESSAGE = """
You are a helpful assistant. You have access to relevant
excerpts from a transcript:

{retrieved_context}

Use these excerpts to answer the user's questions. If the
answer cannot be found in the provided excerpts, say so clearly.
Do not make up information not present in the excerpts.
"""
```

Figure A.29: Grounded QA system message template defining assistant role and constraints

```
from langchain_core.prompts import ChatPromptTemplate

def get_rag_chat_prompt():
    """Create RAG chat prompt template with chat history support."""
    return ChatPromptTemplate.from_messages([
        ("system", RAG_SYSTEM_MESSAGE),
        ("placeholder", "{chat_history}"),
        ("human", RAG_HUMAN_MESSAGE)
    ])
```

Figure A.30: Complete RAG chat prompt construction with history support

```
from langchain_core.messages import HumanMessage, AIMessage

# Example chat history structure
chat_history = [
    HumanMessage(content="What is the project timeline?"),
    AIMessage(content="The project timeline spans 6 months..."),
    HumanMessage(content="Who is responsible for Phase 2?"),
    AIMessage(content="Speaker B is responsible for Phase 2...")
]

# Injected into prompt via placeholder
prompt.invoke({
    "retrieved_context": formatted_context,
    "chat_history": chat_history,
    "question": "When does Phase 2 start?"
})
```

Figure A.31: Chat history structure for multi-turn conversation support

```python
# Streaming invocation for real-time display
for chunk in chain.stream({
    "retrieved_context": formatted_context,
    "chat_history": chat_history,
    "question": user_question
}):
    # Print each token as it arrives
    print(chunk, end="", flush=True)

print()  # Newline after complete response
```

Figure A.32: Streaming response handling for real-time token display

Appendix B

Detailed Evaluation Pipeline Implementation

This appendix collects the implementation-level artefacts that support the evaluation methodology summarized in Chapter 5. It includes prompt templates, intermediate data formats, algorithm illustrations, and supporting benchmark tables that were moved out of the main chapter to keep the final report concise.

B.1 Dataset Generation Artefacts

The evaluation dataset was constructed by grouping raw SRT transcript segments into larger context windows and prompting an LLM to generate grounded question-answer pairs from each window.

```
Chunk 1:
Timestamp: 00:01:30,000 --> 00:01:45,000
Text: "The project timeline spans six months,
       from January to June."

Chunk 2:
Timestamp: 00:01:45,500 --> 00:02:00,200
Text: "We need to complete Phase 1 by the end
       of February."

Chunk 3:
Timestamp: 00:02:00,800 --> 00:02:15,300
Text: "Speaker B is responsible for coordinating
       the deliverables."

           ↓ (Grouping Algorithm)

start_time: 00:01:30,000
end_time: 00:02:15,300
text: "The project timeline spans six months, from
       January to June. We need to complete Phase 1
       by the end of February. Speaker B is
       responsible for coordinating the deliverables."
chunk_indices: [0, 1, 2]

           ↓ (LLM QA Generation)
```

Figure B.1: Example transformation from raw SRT transcript chunks to a grouped window with temporal metadata

```python
def initialize_generator_chain():

    llm = ChatOpenAI(
        model="deepseek-chat",
        openai_api_base=DEEPSEEK_BASE_URL,
        openai_api_key=DEEPSEEK_API_KEY,
        temperature=0.7
    )

    prompt = get_qa_generation_prompt()

    return prompt | llm | StrOutputParser()
```

Figure B.2: LangChain pipe syntax for QA generation chain

```
f"""Based on the following transcript excerpt, generate 2-3 question-answer pairs.

Transcript:
{context}

Requirements:
1. Questions should be specific and answerable from the text
2. Include a mix of detail-oriented and conceptual questions
3. Answers should be concise but complete
4. Questions should be self-contained (no unresolved pronouns)

Output format (JSON without markdown):
[
  {{
    "question": "...",
    "answer": "..."
  }},
  ...
]
"""
```

Figure B.3: QA generation prompt template with requirements and format

The prompt enforces self-contained questions, grounded answers, and valid JSON output. In practice, generating two to three QA pairs per context window provided a useful balance between dataset size and answer quality.

```
[
  {
    "question": "What is the title of the passage?",
    "answer": "The title is 'The North Wind and the Sun'.",
    "source_start": "00:00:00,640",
    "source_end": "00:03:19,180",
    "context": "full text of source context window..."
  }
]
```

Figure B.4: Ground truth JSON format example with timestamps

Field	Type	Description
question	string	Self-contained question derived from context
answer	string	Ground truth answer from source text
source_start	string	Start timestamp in SRT format (HH:MM:SS,mmm)
source_end	string	End timestamp in SRT format
context	string	Full context window text used for generation

Table B.1: Ground truth schema field descriptions

B.2 Metric Implementation Details

Retrieval relevance was determined by checking whether a retrieved chunk overlapped the ground-truth timestamp range associated with each QA pair.

```
def check_retrieval_relevance(
    retrieved_chunk_sources: List[Dict],
    ground_truth_start: str,
    ground_truth_end: str
) -> Tuple[bool, int]:

    gt_start = parse_timestamp(ground_truth_start)
    gt_end = parse_timestamp(ground_truth_end)

    for i, source in enumerate(retrieved_chunk_sources, 1):
        chunk_start = parse_timestamp(source['start_time'])
        chunk_end = parse_timestamp(source['end_time'])

        # Check for overlap
        # Overlap happens if chunk_start < gt_end AND chunk_end > gt_start
        if chunk_start < gt_end and chunk_end > gt_start:
            return True, i

    return False, 0
```

Figure B.5: Interval intersection algorithm for retrieval relevance checking

The same overlap logic supports both Hit Rate and MRR computation by determining whether a relevant chunk exists and, if so, the rank of the first relevant result.

```python
def calculate_f1_score(prediction: str, ground_truth: str) -> float:
    """Calculates F1 score based on token overlap."""
    def normalize_text(text):
        return re.sub(r'\W+', ' ', text.lower()).strip()

    pred_tokens = normalize_text(prediction).split()
    gt_tokens = normalize_text(ground_truth).split()

    if not pred_tokens or not gt_tokens:
        return 0.0
```

Figure B.6: Text preprocessing for token-level comparison

```python
    common_tokens = collections.Counter(pred_tokens) & collections.Counter(gt_tokens)
    num_same = sum(common_tokens.values())

    if num_same == 0:
        return 0.0

    precision = num_same / len(pred_tokens)
    recall = num_same / len(gt_tokens)

    return 2 * (precision * recall) / (precision + recall)
```

Figure B.7: Precision, recall, and harmonic mean calculation with Counter intersection

For lexical evaluation, both the prediction and reference answer were normalized before token-level comparison. This reduces the impact of punctuation and formatting differences while preserving word-level content.

```python
f"""Question: {question}
Ground Truth: {ground_truth}
Generated Answer: {prediction}

Is the Generated Answer semantically correct and consistent with the Ground Truth?
Respond with ONLY one word: "CORRECT" or "INCORRECT".

Response:"""
```

Figure B.8: LLM judge prompt template for semantic correctness evaluation

The semantic judge receives the question, reference answer, and generated prediction, and returns a binary correctness decision that is then parsed into the final accuracy metric.

B.3 Speed Benchmarking Artefacts

The speed evaluation pipeline was separated from the main quality evaluation loop and executed in an interleaved pattern to reduce prompt-caching bias.

```
Query 1: RAG → Pure LLM
Query 2: RAG → Pure LLM
Query 3: RAG → Pure LLM
...
```

Figure B.9: Interleaved execution pattern for unbiased speed benchmarking

Metric	**Description**
Mean	Average query time across all timed queries
Median	Middle value (robust to outliers)
Min	Best-case performance (fastest query)
Max	Worst-case performance (slowest query)
Std Dev	Standard deviation (consistency measure)

Table B.2: Statistical metrics table for latency analysis

Metric	**Formula**	**Interpretation**
Speedup Factor	Pure LLM Mean / RAG Mean	>1: RAG faster, <1: Pure LLM faster, =1: equivalent
Time Difference (s)	Pure LLM Mean - RAG Mean	Absolute time saved per query
Percentage Faster	(Difference / Pure LLM Mean) $\times$ 100	Relative improvement percentage

Table B.3: Comparative metrics between RAG and Pure LLM approaches

Appendix C

Extended Evaluation Examples

This appendix collects worked evaluation examples that support the interpretation in Chapter 5. They are retained here as supplementary diagnostic material rather than part of the main final-report narrative.

C.1 Stage-Separated Diagnostic Scenarios

The evaluation framework separates retrieval quality from generation quality so that different failure modes can be interpreted more precisely.

C.1.1 Retrieval Failure

```
{
  "retrieval_hit": false,
  "retrieval_rank": 0,
  "is_correct": false,
  "f1_score": 0.0
}
```

Figure C.1: Retrieval failure scenario diagnostic example

In this case, the pipeline failed to retrieve any temporally relevant chunk. Such failures suggest that retriever configuration, fusion behavior, or reranking may need further adjustment.

C.1.2 Generation Failure

```
{
  "retrieval_hit": true,
  "retrieval_rank": 1,
  "is_correct": false,
  "f1_score": 0.42
}
```

Figure C.2: Generation failure scenario with successful retrieval

Here, retrieval succeeded but the answer-generation stage focused on the wrong aspect of the evidence. This type of failure is more likely to be addressed through prompt refinement, model choice, or context presentation.

C.1.3 Perfect Performance

```
{
  "retrieval_hit": true,
  "retrieval_rank": 1,
  "is_correct": true,
  "f1_score": 0.87
}
```

Figure C.3: Perfect performance scenario for both retrieval and generation

This case shows the intended behavior of the full pipeline: relevant evidence is retrieved at the top rank and the generated answer is both lexically and semantically aligned with the reference.

C.2 Detailed Worked Examples

C.2.1 Success Case

```
{
  "question": "According to the passage, what did the North Wind and the
               Sun quarrel about?",
  "ground_truth": "The North Wind and the Sun quarreled about which of
                  them was the stronger.",
  "generated_answer": "According to the passage, the North Wind and the
                      Sun quarrelled about which of them was the
                      stronger.",
  "retrieval_hit": true,
  "retrieval_rank": 1,
  "is_correct": true,
  "f1_score": 0.8125,
  "sources": [
    {
      "chunk_id": 1,
      "start_time": "00:00:15,140",
      "end_time": "00:01:31,240",
      "rerank_score": 3.172,
      "preview": "the north wind and the sun had a quarrel about which of
                 them was the stronger..."
    }
  ]
}
```

Figure C.4: Detailed success case example with perfect retrieval and generation

This example illustrates a strong end-to-end result, with the relevant chunk ranked first and the answer judged correct despite a minor spelling variation.

C.2.2 Failure Case

```
{
  "question": "What does the speaker find unusual about the blind box packaging?",
  "ground_truth": "The speaker notes that the box does not say 'Laboo' or
                   'Labu' anywhere, but instead says 'exciting macaron' and
                   'the monsters', and that the top sticker feels tampered
                   with.",
  "generated_answer": "Based on the provided excerpts, the speaker finds that
                       the packaging of the blind box \"smells kind of
                       artificial\" and specifically \"like a carpet\"
                       (Chunk 2).",
  "retrieval_hit": true,
  "retrieval_rank": 1,
  "is_correct": false,
  "f1_score": 0.291
}
```

Figure C.5: Failure case example with retrieval success but generation error

This is a true generation error: the model had access to the relevant evidence but answered a secondary aspect of the question rather than the intended main point.

C.2.3 False Negative Case

```
{
  "question": "What was the hardest decision the speaker had to make?",
  "ground_truth": "The hardest decision was choosing between two job offers:
                   one with NUHS in the memory clinic doing research, and the
                   other at the current place, which they ultimately chose.",
  "generated_answer": "The hardest decision the speaker had to make was
                       choosing between two job offers. One was with a memory
                       clinic doing research, and the other was joining HDB.
                       The speaker had to make this decision within a few days,
                       and in the end, they chose to join HDB, which they are
                       happy with.",
  "retrieval_hit": true,
  "retrieval_rank": 1,
  "is_correct": false,
  "f1_score": 0.585
}
```

Figure C.6: False negative example with more specific generated answer

In this case, the generated answer is more specific than the reference answer while remaining grounded in the transcript. The example highlights a limitation of the current

judging setup, where semantically valid elaboration can still be marked as incorrect.

Bibliography

[1] M. Shao, A. Basit, R. Karri, and M. Shafique, "Survey of different large language model architectures: Trends, benchmarks, and challenges," *IEEE Access*, vol. 12, pp. 188 664–188 706, 2024.

[2] Hostinger. (2025) Llm statistics 2025: Adoption, trends, and market insights. [Online]. Available: https://www.hostinger.com/tutorials/llm-statistics

[3] S. K. Dam, N. Sharma, M. Sahay, and M. Shah, "A complete survey on llm-based ai chatbots," *arXiv preprint arXiv:2406.16937*, 2024.

[4] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.

[5] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[6] NVIDIA. (2023) Mastering llm techniques: Inference optimization. NVIDIA Technical Blog. [Online]. Available: https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization/

[7] K. Kuhn, V. Kersken, B. Reuter, N. Egger, and G. Zimmermann, "Measuring the accuracy of automatic speech recognition solutions," *ACM Transactions on Accessible Computing*, vol. 16, no. 4, pp. 25:1–25:23, 2023.

[8] W. Fan, Y. Ding, L. Ning, S. Wang, H. Li, D. Yin, T.-S. Chua, and Q. Li, "A survey on rag meeting llms: Towards retrieval-augmented large language models," in *Proc. 30th ACM SIGKDD Conf. Knowledge Discovery and Data Mining*, 2024.

[9] Wikipedia. (2024) Mean reciprocal rank. [Online]. Available: https://en.wikipedia.org/wiki/Mean_reciprocal_rank

[10] G. V. Cormack, C. L. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2009, pp. 758–759.

[11] R. Nogueira and K. Cho, "Passage re-ranking with bert," *arXiv preprint arXiv:1901.04085*, 2019.

[12] Otter.ai. (2026) Otter transcribe voice notes. [Online]. Available: https://apps.apple.com/na/app/otter-transcribe-voice-notes/id1276437113

[13] ——. (2026) Ai sales notetaker. [Online]. Available: https://otter.ai/sales-agent

[14] ——. (2026) Ai education notetaker for real-time lecture notes. [Online]. Available: https://otter.ai/education-agent

[15] ——. (2026) Otter features. [Online]. Available: https://otter.ai/features

[16] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[17] Otter.ai Help Center. (2026) Otter AI chat FAQs. [Online]. Available: https://help.otter.ai/hc/en-us/articles/17016733191703-Otter-AI-Chat-FAQs

[18] Anthropic. (2026) Claude code tutorials. [Online]. Available: https://docs.anthropic.com/en/docs/claude-code/tutorials

[19] ——. (2026) Claude code settings. [Online]. Available: https://docs.anthropic.com/en/docs/claude-code/settings

[20] ——. (2026) Claude code team. [Online]. Available: https://docs.anthropic.com/en/docs/claude-code/team

[21] ——. (2026) Manage claude code memory. [Online]. Available: https://docs.anthropic.com/en/docs/claude-code/memory

[22] ——. (2026) Using extended thinking. [Online]. Available: https://support.anthropic.com/en/articles/10574485-using-extended-thinking

[23] Meadow9. (2026) Meadow9. [Online]. Available: https://apps.apple.com/au/app/meadow9/id1474941556

[24] ABAX.AI. (2026) Abax ai | custom speech to text. [Online]. Available: https://www.abax.ai/

[25] ABAX.AI PRIVATE LIMITED. (2026) Abax.ai private limited (202128671g). [Online]. Available: https://www.companies.sg/business/202128671G/ABAX-AI-PRIVATE-LIMITED

[26] Meadow9+. (2026) Meadow9+. [Online]. Available: https://apps.apple.com/us/app/meadow9/id6740014005

[27] C. J. van Rijsbergen, *Information Retrieval*, 2nd ed. London: Butterworths, 1979. [Online]. Available: https://search.worldcat.org/title/Information-retrieval/oclc/4494554

[28] LangChain. (2026) Document. [Online]. Available: https://api.python.langchain.com/en/latest/core/documents/langchain_core.documents.base.Document.html

[29] ——. (2026) Prompt templates. [Online]. Available: https://python.langchain.com/docs/concepts/prompt_templates/

[30] ——. (2026) Output parsers. [Online]. Available: https://api.python.langchain.com/en/latest/core/output_parsers.html

[31] FastAPI. (2026) Fastapi. [Online]. Available: https://fastapi.tiangolo.com/

[32] Pydantic. (2026) Pydantic. [Online]. Available: https://docs.pydantic.dev/

[33] Jina AI. (2026) jina-embeddings-v3. [Online]. Available: https://huggingface.co/jinaai/jina-embeddings-v3

[34] BAAI. (2026) bge-reranker-v2-m3. [Online]. Available: https://huggingface.co/BAAI/bge-reranker-v2-m3

[35] Qwen Team. (2026) Qwen3-14b-base. [Online]. Available: https://huggingface.co/Qwen/Qwen3-14B-Base

[36] ggml-org. (2026) llama.cpp. [Online]. Available: https://github.com/ggml-org/llama.cpp

[37] LangChain. (2026) Chroma vector store. [Online]. Available: https://python.langchain.com/api_reference/chroma/vectorstores/langchain_chroma.vectorstores.Chroma.html

[38] Chroma. (2026) Performance. [Online]. Available: https://docs.trychroma.com/guides/deploy/performance

[39] DeepSeek. (2026) Deepseek api documentation. [Online]. Available: https://api-docs.deepseek.com/

[40] P. E. Black. (2021) double metaphone. National Institute of Standards and Technology. [Online]. Available: https://www.nist.gov/dads/HTML/doubleMetaphone.html

[41] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Dokl. Akad. Nauk SSSR*, vol. 163, no. 4, pp. 845–848, 1965. [Online]. Available: https://www.mathnet.ru/eng/dan31411

[42] LangChain. (2026) Chatprompttemplate. [Online]. Available: https://python.langchain.com/api_reference/core/prompts/langchain_core.prompts.chat.ChatPromptTemplate.html

[43] ——. (2026) Messagesplaceholder. [Online]. Available: https://api.python.langchain.com/en/latest/prompts/langchain_core.prompts.chat.MessagesPlaceholder.html

[44] dateparser. (2026) dateparser documentation. [Online]. Available: https://dateparser.readthedocs.io/en/stable/index.html

[45] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "Ares: An automated evaluation framework for retrieval-augmented generation systems," *arXiv preprint arXiv:2311.09476*, 2023.

[46] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "Ragas: Automated evaluation of retrieval augmented generation," *arXiv preprint arXiv:2309.15217*, 2023.

[47] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-eval: Nlg evaluation using gpt-4 with better human alignment," *arXiv preprint arXiv:2303.16634*, 2023.

[48] OpenAI. (2024) Prompt caching in the api. [Online]. Available: https://openai.com/index/api-prompt-caching/

[49] metaphone. (2026) metaphone. [Online]. Available: https://pypi.org/project/metaphone/

[50] RapidFuzz. (2026) Rapidfuzz usage: fuzz. [Online]. Available: https://rapidfuzz.github.io/RapidFuzz/Usage/fuzz.html

[51] wordfreq. (2026) wordfreq. [Online]. Available: https://pypi.org/project/wordfreq/